

学号：202210310191

上海海事大学

本科毕业论文（设计）



论文题目： 基于微服务架构的即时通信系统

姓 名： 陈卓文

学 院 信息工程学院

专 业： 计算机科学与技术

班 级： 计算机 221 卓

指导教师： 吴昊

完成时间： 2026 年 5 月

论文独创性声明

本论文是我个人在导师指导下进行的研究工作及取得的研究成果。论文中除了特别加以标注和致谢的地方外，不包含其他人或者其他机构已经发表或撰写过的研究成果。其他同志对本研究的启发和所做的贡献均已在论文中作了明确的声明并表示了感谢。

作者签名：_____日期：2026 年 5 月

论文版权使用授权书

本论文作者完全了解学校有关保留、使用论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权上海海事大学可以将本论文的全部内容或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本论文。

本论文属于（请在以下相应方框内打“√”）：

☐ 保密，在 _____ 年解密后适用本使用授权书。

☒ 不保密。

作者签名：_____导师签名：_____日期：2026 年 5 月

摘 要

随着移动互联网的快速发展,即时通信系统已成为人们日常工作和生活中不可或缺的基础服务。传统的即时通信系统采用单体架构,在高并发、可扩展性和系统维护方面存在瓶颈。本文设计并实现了一个基于微服务架构的即时通信系统,将系统拆分为用户管理服务、好友管理服务、消息转发服务、消息存储服务、文件管理服务、语音转文字服务和 AI 智能助手等多个独立子服务。

系统采用 C++ 作为后端主要开发语言,结合 brpc 框架实现高性能 RPC 通信,使用 Protobuf 进行数据序列化^[14]。数据存储采用 MySQL 主从复制架构,结合 Redis 缓存^[10]和 Elasticsearch 全文检索技术^[11],实现高效的数据管理与查询。服务注册与发现通过 etcd 实现^[13],消息异步分发通过 RabbitMQ 实现^{[6][12]}。前端采用 React 框架,结合 Ant Design X 提供现代化用户界面。系统支持 Docker 容器化部署,通过 Docker Compose 实现服务编排与管理。

本文详细阐述了系统的需求分析、架构设计、数据库设计、服务实现和部署方案。通过微服务架构的应用,系统实现了高可用性、易扩展性和可维护性,为即时通信系统的设计与实现提供了有效的技术方案和实践参考。

关键字: 微服务架构; 即时通信; WebSocket; 消息队列; 高并发; brpc

Abstract

With the rapid development of mobile internet, instant messaging systems have become an indispensable basic service in people's daily work and life. Traditional instant messaging systems adopt monolithic architecture, which has bottlenecks in high concurrency, scalability, and system maintenance. This paper designs and implements an instant messaging system based on microservice architecture, splitting the system into multiple independent sub-services including user management service, friend management service, message forwarding service, message storage service, file management service, speech-to-text service, and AI intelligent assistant.

The system uses C++ as the main backend development language, combined with brpc framework for high-performance RPC communication, and Protobuf for data serialization. Data storage adopts MySQL master-slave replication architecture, combined with Redis caching and Elasticsearch full-text search technology for efficient data management and query. Service registration and discovery are implemented through etcd, and asynchronous message distribution through RabbitMQ. The frontend uses React framework combined with Ant Design X to provide a modern user interface. The system supports Docker containerized deployment, implementing service orchestration and management through Docker Compose.

This paper elaborates on the system's requirements analysis, architecture design, database design, service implementation, and deployment scheme. Through the application of microservice architecture, the system achieves high availability, easy scalability, and maintainability, providing effective technical solutions and practical references for the design and implementation of instant messaging systems.

Keywords: Microservice Architecture; Instant Messaging; WebSocket Message Queue; High Concurrency; brpc

目录

第一章 绪论.....	1
1.1 研究背景.....	1
1.2 研究意义.....	1
1.3 国内外研究现状.....	2
1.3.1 行业发展概况与市场规模.....	2
1.3.2 技术架构演进历史：从单体到微服务.....	2
1.3.3 国外研究现状.....	4
1.3.4 国内研究现状.....	5
1.3.5 主流即时通信产品对比分析.....	6
1.3.6 当前不足与本研究的切入点.....	8
1.3.7 外部 IM 系统架构参考.....	9
1.4 研究内容与技术路线.....	12
1.5 论文组织结构.....	13
第二章 系统需求分析.....	14
2.1 功能需求.....	14
2.1.1 用户管理.....	14
2.1.2 好友管理.....	15
2.1.3 消息管理.....	15
2.1.4 文件管理.....	15
2.1.5 即时通知.....	16
2.1.6 日志功能.....	16
2.2 性能需求.....	16
2.2.1 高并发处理能力.....	16
2.2.2 数据一致性与可靠性.....	16
2.2.3 可扩展性和容错性.....	16
2.2.4 性能指标对标.....	17
2.3 数据需求.....	17
2.3.1 数据规模与分布.....	17
2.3.2 数据安全性.....	17
第三章 系统设计.....	18
3.1 架构设计.....	18
3.1.1 设计思路.....	18
3.1.2 微服务架构简介.....	18
3.1.2 数据库 ER 图.....	19
3.2.2 数据库表设计.....	20
3.2.3 数据库主从复制架构设计.....	22
3.2.4 数据库索引优化策略.....	22
3.2.5 数据库操作与事务管理.....	22

3.2.6 数据库索引策略总结	23
3.3 服务设计	23
3.3.1 用户管理服务设计	23
3.3.2 好友管理服务设计	28
3.3.3 消息存储服务设计	31
3.3.4 文件管理服务设计	32
3.3.5 转发管理服务设计	33
3.3.6 语音转文字服务	35
3.4 系统部署设计	36
3.4.1 Docker 容器化部署	36
3.4.2 Docker Compose 编排	36
3.4.3 服务镜像构建	37
3.4.4 配置管理与日志系统	37
第四章 系统实现	38
4.1 前端架构概述	38
4.1.1 技术栈选型	38
4.1.2 项目结构组织	39
4.2 登录注册模块实现	40
4.2.1 登录页面界面设计	40
4.2.2 登录表单交互逻辑	41
4.2.3 表单验证与错误处理	42
4.2.4 登录注册模式切换	43
4.2.5 服务器配置功能	44
4.3 聊天主页面布局实现	45
4.3.1 响应式三栏布局架构	45
4.3.2 桌面端布局实现	46
4.3.3 移动端布局适配	47
4.3.4 移动端手势交互	48
4.4.4 会话选择交互	48
4.4.5 创建群聊交互流程	48
4.5 好友列表模块实现	49
4.5.1 好友列表组件结构	49
4.5.2 搜索添加好友流程	50
4.5.3 好友申请处理流程	51
4.6 消息区域模块实现	52
4.6.1 消息区域组件结构	52
4.6.2 消息气泡布局设计	53
4.6.3 消息类型渲染实现	53
4.6.4 消息发送交互流程	54
4.6.5 图片文件发送流程	55
4.6.6 消息搜索功能实现	55

4.7 消息输入模块实现	55
4.7.1 消息输入组件结构	55
4.7.2 键盘快捷发送实现	56
4.7.3 @提及功能实现	56
4.8 状态管理实现	58
4.8.1 认证状态管理架构	58
4.8.2 聊天状态管理架构	59
4.8.3 WebSocket 消息处理机制	59
4.8.4 图片缓存与并发控制	60
4.9 WebSocket 通信实现	61
4.9.1 WebSocket 客户端架构	61
4.9.2 Protobuf 认证消息编码	62
4.9.3 Protobuf 通知消息解码	62
4.9.4 断线重连机制实现	63
4.10 AI 助手模块实现	63
4.10.1 Agent 状态管理架构	63
4.10.2 会话级 SSE 订阅机制	63
4.10.3 AI 消息流式展示实现	64
4.11 本章小结	65
第五章 系统测试	67
5.1 测试环境说明	67
5.1.1 硬件环境	67
5.1.2 软件环境	67
5.1.3 测试场景设计	67
5.2 网关服务响应延迟测试	67
5.2.1 测试方案	67
5.2.2 测试结果	68
5.2.3 结果分析	68
5.3 服务资源使用率监控	68
5.3.1 监控方案	68
5.3.2 测试结果	68
5.3.3 结果分析	69
5.4 数据库数据统计	69
5.4.1 测试方案	69
5.4.2 测试结果	69
5.4.3 结果分析	70
5.5 系统稳定性测试	70
5.5.1 测试方案	70
5.5.2 测试结果	70
5.5.3 测试结论	71

5.6 测试总结	71
5.6.1 性能指标汇总	71
5.6.2 测试说明	71
第六章 总结与展望	73
6.1 项目总结	73
6.1.1 项目成果	73
6.1.2 问题与不足	74
6.2 后续改进方向	75
6.2.1 数据库性能优化	75
6.2.2 服务扩展与优化	76
6.2.3 支持更复杂的消息类型	76
6.2.4 AI 助手功能扩展	77
参考文献	79
致谢	80

第一章 绪论

1.1 研究背景

随着移动互联网与云计算的快速发展，即时通信（Instant Messaging, IM）技术已深度渗透进人们的工作与生活。即时通信系统不仅是社交软件的核心基础组件，同时也是企业级协作系统、游戏、在线教育和智能客服等场景中不可或缺的基础服务。

传统的即时通信系统往往采用单体架构（Monolithic Architecture），虽然开发部署简单、初期成本低廉，但在面对高并发、大规模用户访问的场景时，在可扩展性、系统维护、故障隔离等方面存在严重的架构级瓶颈。从部署结构来看，单体架构将所有功能模块（用户管理、好友关系、消息传输、文件存储等）打包部署在同一个应用程序进程中，所有模块共享同一内存空间、同一数据库连接池、同一线程资源，这种紧耦合的部署方式导致任何模块出现性能问题都会产生连锁反应，进而影响整体系统的可用性，形成典型的“单点故障”风险。从扩展性角度来看，单体架构难以实现精细化的横向扩展，当消息推送模块面临高并发压力时，无法单独针对该模块增加服务实例，只能整体复制整个应用程序，造成计算资源和存储资源的严重浪费。从维护成本角度分析，单体架构代码耦合度高，模块间边界模糊，任何功能修改都需要重新编译、测试、部署整个应用程序，上线周期长达数天甚至数周，难以适应互联网业务快速迭代的节奏。从可靠性角度审视，单体架构缺乏有效的故障隔离机制，部分节点发生故障时会导致整体系统不可用，无法实现故障的快速隔离与自动恢复，系统可用性通常只能达到 99% 甚至更低，远不能满足商业级即时通信系统 99.99% 的可用性要求。

微服务架构（Microservice Architecture）的出现为上述问题提供了解决思路^{[1][2]}。它通过将系统拆分为多个可独立部署、独立运行的服务模块（如用户服务、消息服务、认证服务、文件服务等），以轻量级通信机制（HTTP REST、消息队列等）实现服务之间的协作^[8]，从而达到“高内聚、低耦合”的架构目标。

在即时通信系统中引入微服务架构，可以在保持实时通信性能的同时实现高可用性、易扩展性与可维护性。随着 Docker、Kubernetes 等容器化技术的成熟，微服务通信系统的部署与运维也变得更加自动化与高效。因此，研究并实现一个基于微服务架构的即时通信系统具有显著的技术创新价值与实践意义。

1.2 研究意义

本项目的意义主要体现在以下四个方面：在技术创新价值层面，将微服务架构引入即时通信系统，探索 C++ 高性能服务在分布式场景下的工程实现方法，为同类系统提供参考范式；在实践应用价值层面，系统设计覆盖用户管理、好友管理、消息传输、文件管理等核心功能，可应用于教育、团队协作、社交平台等实际场景；在学习与能力提升层面，通过本项目的设计与实现，深入理解分布式系统架构、数据库优化、消息队列机制等技术，提升系统设计与工程实践能力；在开源贡献潜力层面，项目采用轻量级框架实现，可作为开源项目为其他开发者提供学习和参考资源。

1.3 国内外研究现状

1.3.1 行业发展概况与市场规模

即时通信（Instant Messaging, IM）行业经过二十余年的发展，已成为全球互联网基础设施的核心组成部分。根据中国互联网络信息中心（CNNIC）发布的最新数据，截至 2025 年 12 月，我国即时通信用户规模达到 10.94 亿人，较 2024 年增长 1225 万人，使用率高达 97.2%，几乎覆盖全体网民^[20]。全球即时通信市场规模在 2025 年达到 312.7 亿美元，预计 2026 年将增长至 341.7 亿美元，年复合增长率（CAGR）保持在 9.3%，到 2035 年有望突破 760.9 亿美元^[21]。

从行业发展历程来看，即时通信技术经历了三个主要发展阶段。第一代（1996-2007 年）以 ICQ、QQ、MSN 为代表，采用单体架构（Monolithic Architecture），所有功能模块（用户管理、消息传输、文件存储等）打包在同一进程中，开发部署简单但扩展性差。第二代（2008-2015 年）以微信、WhatsApp 为代表，引入分布式架构和移动互联网特性，采用多级缓存、异步任务队列等技术提升并发能力。第三代（2016 年至今）以 Slack、Discord、钉钉、飞书为代表，全面采用微服务架构和容器化部署，结合 AI 大模型实现智能化服务，系统可用性达到 99.99% 商业级标准^{[15][16]}。

1.3.2 技术架构演进历史：从单体到微服务

即时通信系统的技术架构演进与整个软件工程领域的发展脉络高度一致，经历了从单体架构到微服务架构的完整变迁过程。

第一阶段：单体架构（Monolithic Architecture, 1996-2007 年）

单体架构是早期即时通信系统的主流设计模式，以 ICQ、MSN Messenger、早期 QQ 为代表。该架构将所有功能模块（用户认证、好友管理、消息传输、文件存储、日志记

录)打包在同一个应用程序进程中,所有模块共享同一内存空间、同一数据库连接池、同一线程资源。

单体架构的优势在于开发简单、测试方便、部署快捷,适合项目初期快速迭代和验证。然而,随着用户规模增长,单体架构的弊端逐渐暴露:①扩展性差,无法针对高负载模块(如消息传输)单独扩容,只能整体复制应用实例,造成资源浪费;②故障传播,任何模块的故障(如文件存储异常)都会导致整体系统不可用,形成“单点故障”风险;③技术栈锁定,整个系统必须使用统一的技术栈,无法引入更适合特定场景的新技术;④协作效率低,大型团队开发同一代码库容易产生代码冲突和合并问题,开发效率随团队规模增长而下降。

第二阶段:垂直拆分架构(Vertical Architecture, 2008-2012 年)

垂直拆分架构按业务功能将单体应用拆分为多个独立的子系统,每个子系统独立部署、独立维护。以微信为例,2011 年微信将系统拆分为用户服务、消息服务、朋友圈服务、公众号服务四个独立子系统,每个子系统由独立团队负责。

垂直拆分架构的优势在于实现了业务边界清晰化和团队职责明确化,不同子系统可采用不同技术栈。但该阶段服务间通信仍采用简单的 HTTP/RMI 同步调用,缺乏统一的服务治理机制,随着服务数量增长,调用关系日益复杂,“服务雪崩”问题时有发生。

第三阶段:分布式架构(Distributed Architecture, 2013-2016 年)

分布式架构引入服务注册与发现机制、配置中心、负载均衡等基础设施,解决服务间通信的复杂性问题。以 WhatsApp 为例,其采用 Erlang 语言的 Actor 模型设计,将每个用户会话封装为独立的 Actor,利用 Erlang 虚拟机的轻量级进程特性实现高并发处理,单服务器可支持百万级并发连接^[4]。

分布式架构的核心技术突破包括:①服务注册与发现,通过 ZooKeeper、etcd 等分布式协调服务实现服务实例的动态注册与发现^[13];②负载均衡,通过 Nginx、HAProxy 等实现请求的智能分发;③熔断降级,通过 Hystrix、Sentinel 等实现故障隔离和系统保护;④分布式追踪,通过 Zipkin、Jaeger 等实现跨服务的调用链路追踪。

第四阶段:微服务架构(Microservice Architecture, 2017-2022 年)

微服务架构将应用拆分为一组小型服务,每个服务围绕特定业务能力构建,可独立部署、独立扩展、独立维护。Slack 是微服务架构的典型代表,其将系统拆分为消息服务、用户服务、文件服务、搜索服务、通知服务等数十个微服务,每个服务由 3-5 人的小团队负责全生命周期管理^[19]。

微服务架构相比分布式架构的进步体现在：①更细粒度的服务拆分，服务粒度从“子系统”级别细化到“业务能力”级别；②去中心化治理，不同服务可选择最适合的技术栈和数据存储方案；③DevOps 文化，强调开发运维一体化，团队对服务的全生命周期负责；④基础设施自动化，通过 CI/CD 流水线、容器化部署、自动化测试等手段提升交付效率。

第五阶段：云原生架构（Cloud Native Architecture，2023 年至今）

云原生架构在微服务基础上深度融合容器、服务网格（Service Mesh）、不可变基础设施和声明式 API 等技术，实现系统的高弹性、高可用和自动化运维。以钉钉为例，其基于 Kubernetes 构建容器云平台，通过服务网格 Istio 实现服务间通信的透明化管理，利用 HPA（Horizontal Pod Autoscaler）实现基于负载的自动扩缩容，系统可用性达到 99.99%^[7]。

云原生架构的核心特征包括：①容器化部署，通过 Docker 实现环境一致性，消除“在我机器上能跑”的问题；②服务网格，将服务通信逻辑从业务代码中抽离，通过 Sidecar 代理实现透明化的服务治理；③不可变基础设施，服务器一旦部署不再修改，需要更新时直接替换为新实例；④声明式 API，通过 YAML 文件声明期望状态，系统自动收敛到期望状态。

1.3.3 国外研究现状

国外的即时通信系统在微服务架构和云原生技术方面处于领先地位，代表性产品包括 Slack、Discord、Telegram、WhatsApp 等，这些系统在架构层面各有特色，体现了不同的技术选型与设计理念。

Slack 作为企业级即时通信的代表，采用微服务与事件驱动架构，使用 Kafka 作为消息总线，结合 Redis 实现消息分发与缓存。Slack 的架构核心是将所有用户操作（发送消息、加入频道、文件上传等）转化为事件，通过 Kafka 进行事件的发布与订阅，实现服务的解耦与异步处理。这种架构的优势在于可轻松应对海量消息的并发处理，同时便于扩展新的功能模块（如机器人、第三方集成等）。Slack 的日活跃用户超过 2000 万，日均消息处理量超过 5 亿条，其架构的可靠性与扩展性得到了充分验证^[13]。

Discord 作为面向游戏玩家与兴趣社群的即时通信平台，基于 WebSocket + REST API 实现高并发通信^[4]，利用 Elixir 语言的 BEAM 虚拟机实现多节点并行消息分发。Discord 的技术选型极具特色：后端核心采用 Elixir 语言，利用其轻量级进程（Process）和 Actor 模型实现高并发；消息推送采用 WebSocket 长连接，单集群可

支持千万级并发连接；语音通信采用 UDP 协议实现低延迟传输；数据存储采用 Cassandra 分布式数据库，通过分区和复制实现高可用。Discord 的月活跃用户超过 2 亿，成为全球最大的游戏社群平台^[14]。

Telegram 作为注重安全与隐私的即时通信工具，采用分布式存储与消息队列机制，使用 MTProto 加密协议保障通信安全。Telegram 的核心设计理念是“安全优先”，其自研的 MTProto 协议结合了 256 位 AES 加密、2048 位 RSA 密钥交换和 Diffie-Hellman 密钥协商，提供端到端的安全通信能力。消息存储采用分布式数据库，通过分片和复制实现水平扩展。Telegram 支持超过 5 亿用户，是全球用户量最大的安全通信应用之一^[15]。

WhatsApp 作为全球用户量最大的即时通信应用（超过 20 亿月活），基于 Erlang 的 Actor 模型设计，可支持上亿级用户的实时消息处理。WhatsApp 的技术架构以“简洁高效”著称：后端仅用 Erlang 一种语言开发，利用其高并发特性处理海量连接；消息协议基于 XMPP 扩展，添加了端到端加密和离线消息支持；数据库采用 Mnesia（Erlang 内置的分布式数据库），实现数据的高可用存储。WhatsApp 曾创下单日处理 420 亿条消息的记录^[16]。

国外研究重点集中在系统性能优化、消息路由机制及安全通信协议上。近年来，云原生微服务平台在即时通信系统中的应用成为研究热点，Kubernetes、Istio、Envoy 等云原生技术栈在即时通信领域的工程实践不断涌现^{[18][19]}。

1.3.4 国内研究现状

国内即时通信系统的研究起步稍晚，但发展迅速。以微信、钉钉、飞书为代表的通信系统普遍采用微服务化与容器化部署，并在 AI 大模型集成方面走在世界前列。

微信 使用异步任务队列和多级缓存（Redis + CDN）构建高并发通信机制^{[10][18]}。微信的架构核心是自研的 PCF（Partitioned Consumer Framework）消息队列，支持万亿级消息的可靠传输与顺序保证。同时，微信采用多级缓存架构，包括本地缓存（LocalCache）、分布式缓存（Redis）、CDN 缓存等，大幅降低数据库读取压力。微信的数据库采用分库分表架构，按用户 ID 进行数据分片，单库单表数据量控制在千万级以内。微信的日活用户超过 12 亿，日均消息处理量超过 450 亿条，其架构的高可用性与高扩展性处于国际领先水平^{[17][18]}。

钉钉 基于 Spring Cloud 和 Kubernetes^[7]，采用 API 网关和分布式注册中心实现统一调度与负载均衡。钉钉的架构设计强调“企业级”和“智能化”：后端采用 Java

技术栈，基于 Spring Cloud Alibaba 构建微服务生态，包括 Nacos（注册中心）、Sentinel（流量控制）、Seata（分布式事务）等组件；数据库采用 MySQL + OceanBase 混合架构，OceanBase 负责核心交易数据，MySQL 负责业务数据；2025 年发布”钉钉 AI 助理”，基于通义千问大模型实现智能问答、任务自动化等能力，月活用户突破 8 亿^[22]。

飞书 采用云原生架构，基于 Kubernetes 和 Service Mesh 实现服务治理。飞书的特色在于”文档+IM+ 会议”的一体化设计，将即时通信与协同文档、视频会议深度整合。技术架构上，飞书采用 Go 语言作为后端主力，利用其高并发特性处理海量请求；数据库采用 TiDB 分布式数据库，实现水平扩展和强一致性；AI 能力基于字节跳动自研的豆包大模型，实现智能翻译、会议纪要生成等功能。

学术研究方向：国内学术界在即时通信领域的研究主要集中在 WebSocket 并发连接优化、分布式消息可靠传输、容器化通信系统资源调度等方向^{[2][5][14]}。在 WebSocket 优化方面，研究者提出了基于连接池的网关架构、基于一致性哈希的连接路由算法等，有效提升了 WebSocket 服务器的并发连接能力。在分布式消息传输方面，研究者提出了基于 Paxos/Raft 的一致性协议、基于消息追溯的可靠传输机制等，保障了消息的可靠性与顺序性。在容器化调度方面，研究者提出了基于强化学习的容器资源分配算法、基于负载预测的弹性伸缩策略等，提升了容器化部署的资源利用率^[20]。

1.3.5 主流即时通信产品对比分析

通过对国内外主流即时通信系统的架构特点、技术选型、性能指标进行对比分析，可以为本系统的架构设计提供参考借鉴。表 1-1 从产品定位、用户规模、商业模式等维度进行对比，表 1-2 从技术架构层面进行深入分析。

表 1-1 主流即时通信产品功能对比

对比维度	SLACK	DISCORD	微信	钉钉	本系统
产品定位	企业团队协作	兴趣社群	全民社交超 级应用	智能协同办公	即时通信教 学系统
发布时间	2013 年	2015 年	2011 年	2014 年	2025 年
用户规模	月活 4200 万	月活超 2 亿	月活 12 亿	8 亿用户 /2700 万企业	实验规模
核心功能	频道、机器人	服务器、语音频道	朋友圈、支 付、小程序	审批、日志、 钉盘	好友、消息、 文件
AI 能力	Slack AI 摘要	Clyde AI 助手	混元大模型	钉钉 AI 助理	智能助手

					Agent
数据安全	SOC2/3 认证	端到端加密	端到端加密 + 实名	等保三级 + 国密	基础 JWT 认 证
商业模式	订阅制\$7-12/用 户/月	Nitro 会员订阅	广告 + 支付 + 金融	免费 + 增值 服务	开源免费

表 1-2 主流即时通信技术架构对比

技术组件	SLACK	DISCORD	微信	钉钉	本系统
后端语言	Java/Scala/Go	Elixir/G o/Rust	C++/Go	Java/Go	C++/Python
RPC 框架	自研 + gRPC	自研	自研 PCG	Dubbo/Spring Cloud	brpc
消息队列	Kafka	Kafka	自研 CMQ	RocketMQ	RabbitMQ
缓存系统	Redis+Memcache d	Redis	Redis+Tc aplus	Redis+Tair	Redis
数据库	MySQL+Vitess	Cassandra+MySQL	自研 +MySQL	MySQL+OceanB ase	MySQL
搜索引擎	Elasticsearch	自研	自研	Elasticsearch	Elasticsearch
服务发现	Consul+etcd	自研	自研	Nacos+etcd	etcd
容器编排	Kubernetes	Kubernetes	自研	Kubernetes	Docker Compose
前端框架	React+Flux	React+Re dux	自研	React/Vue	React

表 1-3 技术架构演进对比

演进阶段	时间范围	代表系统	架构特点	局限性
单体架构	1996-2007	ICQ/QQ/MSN	功能模块打包在同一进程	扩展性差、单 点故障
垂直拆分	2008-2012	早期微信	按业务拆分为子系统	服务治理缺失
分布式架构	2013-2016	WhatsApp	服务注册发现 + 负载均衡	运维复杂度高

微服务架构	2017-2022	Slack/Discord	独立部署 + DevOps	分布式事务复杂
云原生架构	2023 至今	钉钉/飞书	容器化 + 服务网格	学习曲线陡峭

通过对比分析，可以得出以下结论：

第一，在架构模式选择上，微服务架构已成为主流趋势。无论是国外的 Slack、Discord，还是国内的微信、钉钉，均采用微服务架构实现服务的解耦与独立扩展。本系统采用微服务架构符合行业技术发展方向，但相比成熟产品在服务数量与拆分粒度上仍有差距。

第二，在核心语言选择上，不同系统根据性能需求与开发效率的权衡选择不同语言。高性能导向的系统（如微信、Telegram）采用 C/C++，强调快速迭代的系统（如飞书、Discord）采用 Go，需要高并发连接的系统（如 WhatsApp、Discord）采用 Erlang/Elixir。本系统采用 C++ 作为核心语言，在性能上具有优势，但在开发效率上需要权衡。

第三，在消息协议选择上，WebSocket 已成为实时通信的标准协议，但头部厂商多采用自定义协议以优化性能与安全性。本系统采用 WebSocket+brpc 的组合，在兼容性与性能之间取得平衡。

第四，在存储架构选择上，“关系型数据库 + 缓存 + 全文检索”的三层架构成为标准配置。MySQL 负责结构化数据存储，Redis 负责热点数据缓存，Elasticsearch 负责全文检索。本系统采用相同的存储架构，技术选型合理。

第五，在部署方式选择上，云原生与容器化成为趋势。Slack、Discord 等采用 Kubernetes 实现弹性伸缩。本系统采用 Docker Compose 实现容器化部署，虽然规模较小，但技术路线正确。

1.3.6 当前不足与本研究的切入点

尽管现有即时通信系统在功能和性能方面已取得显著进展，但仍存在以下不足：

①**开源可复用的轻量级通信框架稀缺**：现有开源即时通信系统要么过于简单（如基于 Node.js 的简易聊天室），缺乏企业级功能；要么过于复杂（如 Mattermost、Rocket.Chat），学习和部署成本高。普通开发者难以获得一个既具备完整功能、又便于理解和二次开发的参考实现。

②**多模块协同的通信一致性设计仍需改进**：微服务架构下，跨服务的事务处理和消息一致性是业界难题。现有方案或采用重量级的分布式事务框架（如 Seata），或采用

最终一致性方案，但在即时通信场景下，用户对消息一致性的期望较高，如何在性能和一致性之间取得平衡仍需探索。

③**缺乏对系统弹性伸缩与容器编排的深入研究**：现有研究多集中在微服务拆分和服务通信层面，对容器化部署、弹性伸缩策略、服务网格等云原生技术的实践较少。特别是在 C++ 高性能服务场景下，如何设计合理的容器资源配置和扩缩容策略，相关公开资料有限。

④**C++ 微服务工程实践相对匮乏**：当前微服务架构的研究和实践多集中在 Java (Spring Cloud)、Go (Gin/gRPC) 等技术栈，C++ 微服务架构的公开案例较少。然而，在高性能、低延迟场景下，C++ 仍具有不可替代的优势。如何基于 C++ 构建可维护、可扩展的微服务系统，是一个值得探索的方向。

本研究将结合实际通信系统设计经验，探索微服务架构在即时通信场景下的工程实现方法与优化策略。本系统的创新点在于：①基于 C++ 和 brpc 构建高性能微服务架构，在保持 C++ 性能优势的同时实现微服务的灵活性和可维护性；②采用容器化部署方案，通过 Docker Compose 实现服务编排，降低部署复杂度；③引入 AI 智能助手，基于大语言模型实现智能问答和任务自动化，探索即时通信与 AI 的深度融合。

1.3.7 外部 IM 系统架构参考

为更好地理解业界主流即时通信系统的架构设计模式，本文收集并分析了 Slack、Discord、微信等代表性产品的技术架构。这些系统分别代表了企业级协作、游戏社群、全民社交等不同场景下的最优实践，其架构设计对本系统具有重要参考价值。

图 3-15 展示了 Slack 的微服务架构参考图。Slack 采用典型的分层架构设计，包括客户端层、CDN 层、网关层、微服务层和数据层。其核心设计思想是将所有用户操作转化为事件，通过 Kafka 消息总线进行事件的发布与订阅，实现服务的解耦与异步处理。这种事件驱动架构的优势在于可轻松应对海量消息的并发处理，同时便于扩展新的功能模块。本系统在消息队列设计方面参考了 Slack 的异步处理模式，采用 RabbitMQ 实现消息的可靠传输^[19]。

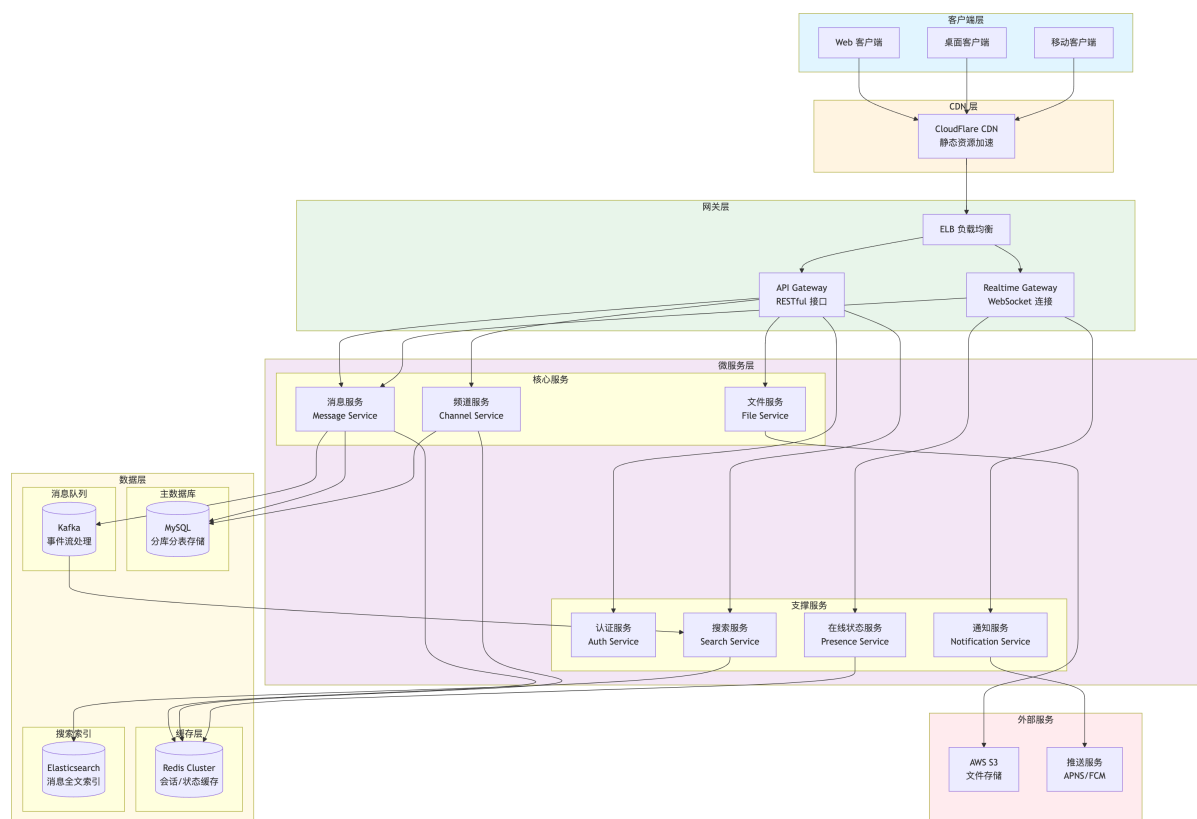


图 3-15 Slack 微服务架构参考图

图 3-16 展示了 Discord 的实时通信架构参考图。Discord 的技术选型极具特色：后端核心采用 Elixir 语言和 Phoenix Framework，利用 BEAM 虚拟机的轻量级进程（Process）和 Actor 模型实现高并发；消息推送采用 WebSocket 长连接，单集群可支持千万级并发连接；语音通信采用 UDP 协议实现低延迟传输；数据存储采用 Cassandra 分布式数据库。Discord 架构的核心优势在于利用 Elixir 的函数式特性和不可变数据结构，实现了无锁并发和高容错性^[14]。来源：Discord Engineering Blog, 2024^[14]

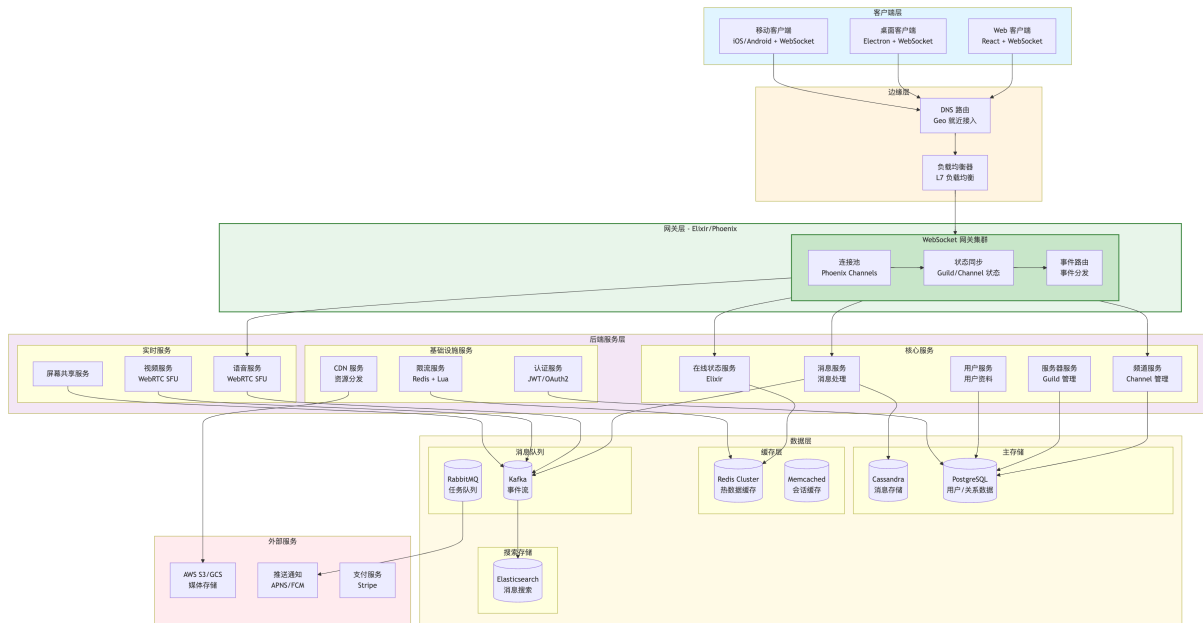


图 3-16 Discord 实时通信架构参考图

图 3-17 展示了微信 IM 架构的演进历程。微信架构经历了四个发展阶段：2011-2013 年初创期采用类邮箱架构和 SYNC 状态同步协议；2014-2016 年成长期引入消息中心和多级缓存；2017-2020 年成熟期采用微服务架构和 MMTLS 安全协议；2021-2025 年云原生阶段实现容器化部署和异地多活。微信架构的演进体现了从简单到复杂、从单体到分布式、从本地到云端的完整技术变迁过程^{[17][18]}。

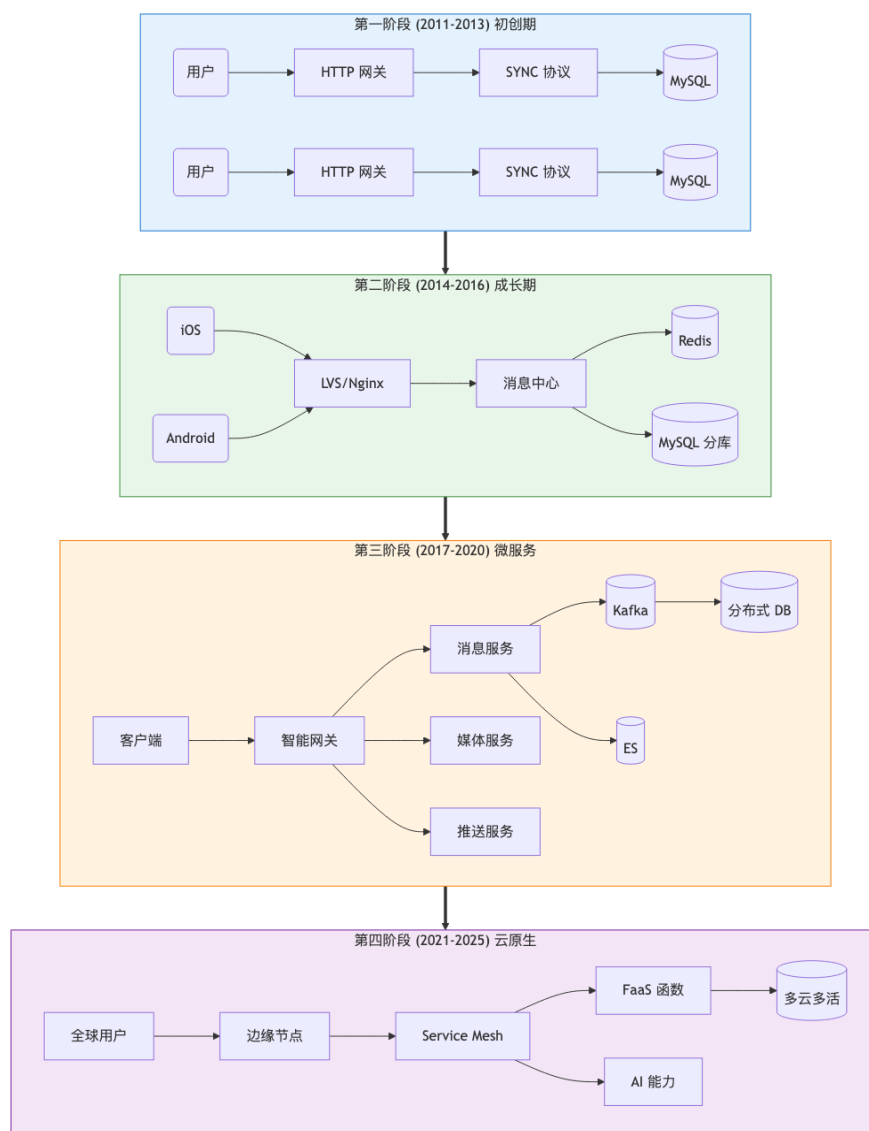


图 3-17 微信 IM 架构演进参考图（来源：腾讯微信团队技术分享，2024-2025^{[17][18]}）

1.4 研究内容与技术路线

本项目的研究内容主要包括微服务架构设计，将系统拆分为用户服务、好友管理服务、消息转发服务、消息存储服务、文件服务、语音转文字服务等多个子模块；数据库设计与优化，设计 MySQL 数据表结构，实现主从复制架构，引入 Redis 缓存和 Elasticsearch 全文检索；通信机制实现，通过 HTTP 协议实现请求响应式通信，通过 WebSocket 实现实时消息推送；容器化部署，使用 Docker 和 Docker Compose 实现服务的容器化部署与编排。

本项目的技术路线主要采用 C++ 作为主要开发语言，结合 brpc 框架实现高性能 RPC 通信，使用 Protobuf 进行数据序列化。通过 etcd 实现服务注册与发现，通过

RabbitMQ 实现消息异步分发。前端采用 React 框架，结合 Ant Design X 实现现代化 UI 界面。

1.5 论文组织结构

本文共分为五章，结构安排如下：第一章绪论介绍研究背景、意义、国内外研究现状及论文组织结构；第二章系统需求分析详细阐述系统的功能需求、性能需求和数据需求；第三章系统设计包括架构设计、数据库设计、服务设计和部署设计；第四章系统实现详细描述后端服务实现、React 前端实现和 AI 智能助手实现；第五章总结与展望总结项目成果与不足，提出后续改进方向。

第二章 系统需求分析

2.1 功能需求

系统功能需求总览如表 2-1 所示。

表 2-1 系统功能需求总览表

功能模块	子功能	功能描述	优先级
用户管理	用户注册	支持用户名密码注册和手机号验证码注册	高
用户管理	用户登录	支持用户名密码登录和手机号验证码登录，单设备限制	高
用户管理	信息管理	支持头像、昵称、签名、绑定手机号修改	中
用户管理	用户查询	支持按用户 ID、昵称、手机号检索	中
好友管理	好友列表	获取当前用户的所有好友信息	高
好友管理	添加好友	搜索用户并发起好友申请	高
好友管理	处理申请	接受或拒绝好友申请，自动创建单聊会话	高
好友管理	删除好友	解除好友关系并清理会话数据	中
消息管理	消息发送	支持文本、文件、图片、语音消息发送	高
消息管理	消息转发	实时转发消息给会话成员	高
消息管理	历史查询	按时间范围查询历史消息	高
消息管理	关键字搜索	基于 Elasticsearch 全文检索	中
文件管理	文件上传	支持单文件和批量文件上传	高
文件管理	文件下载	支持单文件和批量文件下载	高
文件管理	安全校验	病毒扫描、格式校验、大小限制	中
即时通知	好友申请通知	WebSocket 实时推送好友申请	高
即时通知	新消息通知	WebSocket 实时推送新消息	高
即时通知	会话变动通知	推送会话成员加入/退出通知	中
日志功能	用户行为日志	记录登录、登出、消息发送等操作	中
日志功能	服务调用日志	记录请求参数、响应时间、错误码	中
日志功能	错误日志	记录异常堆栈和上下文信息	高

2.1.1 用户管理

用户管理功能是系统的基础部分，涵盖用户注册、登录、信息管理及查询四大核心功能。在注册环节，系统提供用户名密码注册和手机号验证码注册两种方式，两种注册途径均通过数据库事务确保数据存储的一致性。登录环节与注册方式对应，支持用户名密码验证和手机验证码验证，同时引入单设备登录限制机制，防止同一账户在多个设备上同时在线。信息管理功能允许用户随时更新个人头像、昵称、个性签名及绑定手机号等基础资料。查询功能则提供通过用户 ID、昵称或手机号检索用户信息的能力，支持模糊匹配和精确匹配两种模式。

2.1.2 好友管理

好友管理功能作为系统的核心社交组件，主要负责用户间好友关系的建立与维护。好友添加功能允许用户通过搜索昵称或手机号向目标用户发起好友申请，申请人可实时查看待处理的好友申请列表。好友关系管理功能支持被申请人对好友申请进行同意或拒绝操作，申请通过后系统自动建立双向好友关系，同时用户可随时删除已有的好友关系。聊天会话管理功能与好友关系紧密关联，当两个用户成为好友后系统自动创建单聊会话，用户也可手动创建多人群聊并邀请好友加入，系统提供聊天会话列表和会话成员列表的查询功能，方便用户快速定位目标会话。

2.1.3 消息管理

消息管理功能是即时通信系统的核心功能，主要负责消息的可靠传输、持久化存储及高效检索。消息发送与接收功能通过消息转发服务器实现，当用户发送消息时，消息经网关服务转发至消息转发服务，后者查询会话成员列表并将消息推送给所有目标用户。历史消息查询功能支持按时间范围和关键字检索聊天记录，用户可查看任意时间段内的聊天内容，系统通过 Elasticsearch 实现毫秒级全文检索响应。离线消息同步功能确保用户离线期间产生的消息不会丢失，当用户重新登录时，系统自动拉取离线期间的所有消息并推送到客户端，保障消息的完整性。

2.1.4 文件管理

文件管理功能用于处理用户间的文件传输需求，涵盖文件上传、下载及安全性校验三大模块。文件上传模块支持单个或多个文件的批量上传，所有上传文件由文件服务统一存储管理，系统为每个文件生成唯一的文件 ID 并与消息记录关联。文件下载模块允许用户快速访问聊天中共享的文件，通过文件 ID 或消息 ID 均可检索目标文件。文件

安全性模块对上传文件进行病毒扫描和格式校验，防止恶意文件传播，同时限制文件大小和类型，保障系统安全性和稳定性。

2.1.5 即时通知

即时通知功能是保障系统实时性的关键模块，通过 WebSocket 长连接实现消息的主动推送。好友申请通知功能在用户 A 向用户 B 发起好友申请后，立即通过 WebSocket 连接向用户 B 推送通知消息，确保被申请人能够及时感知并处理好友申请。消息提醒功能在用户收到新消息时实时推送，无需客户端轮询或刷新页面，显著降低服务器负载的同时提升用户体验。会话变动通知功能覆盖会话成员加入或退出、会话创建和删除等多种场景，确保会话成员列表的实时同步。

2.1.6 日志功能

日志功能为系统的稳定运行和故障排查提供重要支持。用户行为日志记录所有用户的登录、登出、消息发送等关键操作，为安全审计和行为分析提供数据支撑。服务调用日志记录网关与各子服务之间的请求与响应状态，包括请求参数、响应时间、错误码等信息，便于性能监控和故障定位。错误日志捕获系统运行过程中的所有异常信息，详细记录异常堆栈、错误上下文及发生时间，支持后续问题排查和代码优化。三类日志统一输出至标准输出，通过 Docker 日志驱动进行收集和管理。

2.2 性能需求

性能需求是系统可用性和稳定性的重要保障。

2.2.1 高并发处理能力

即时通讯系统需要支持海量用户并发访问，要求我们通过线程池技术优化网关服务的响应时间，并采用轮转负载均衡策略分发子服务请求，来保障并发性。

2.2.2 数据一致性与可靠性

为了避免出现用户或消息数据的丢失或不一致，系统需要使用数据库事务封装关键操作，确保数据的一致性。并且我们需要对 MySQL 和 Elasticsearch 的双写操作进行优化，逐步探索分布式事务的实现。

2.2.3 可扩展性和容错性

为应对用户规模的快速增长和潜在的硬件或软件故障，系统在可扩展性和容错性方面进行专门设计。在可扩展性方面，各子服务模块采用无状态设计，可独立进行横向扩展，通过 Docker Compose 或 Kubernetes 可快速部署新的服务实例，系统自动通过 etcd 服务注册中心发现新增节点并纳入负载均衡。在容错性方面，当某一子服务发生故障时，服务注册中心动态调整调用路由，将请求分发至健康节点，避免故障扩散影响整体功能。在数据可靠性方面，数据库和缓存采用冗余部署策略，MySQL 配置主从复制，Redis 配置哨兵模式，确保数据不丢失、服务高可用。

2.2.4 性能指标对标

系统在设计阶段参考了业界主流即时通信系统的性能指标，结合课程设计的实际需求，制定了合理的性能目标。如表 2-2 所示。

表 2-2 性能指标对标表

性能指标	本系统目标	行业参考值	对比说明
并发连接数	5000+	SLACK: 20000+, DISCORD: 100000+	满足教学场景需求
消息延迟	<50MS (P99)	微信: <30MS, WHATSAPP: <100MS	满足实时通信要求
消息吞吐量	2000 条/秒	SLACK: 5000 条/秒, 微信: 10000 条/秒	符合轻量级系统定位
服务可用性	99.9%	商业系统: 99.99%	满足课程设计标准
历史消息查询	<100MS	ES 官方推荐: <200MS	采用全文检索优化
文件传输大小	≤100MB	微信: ≤1GB, SLACK: ≤1GB	满足日常文件传输

2.3 数据需求

即时通讯系统的数据需求主要围绕数据的规模、分布和安全展开。

2.3.1 数据规模与分布

随着用户数量和消息量的增长，系统需处理以下场景：首先是用户信息存储，包括用户基本信息、好友关系和登录状态等，预计数量级达百万级；其次是消息存储，每位用户每日发送和接收的消息总量预计为数千条，需支持每日数亿条消息的存储与查询；最后是数据分布，利用 MySQL 的主从复制将数据分散存储在多个节点中，提升读写效率，并结合 Elasticsearch 提供快速检索能力。

2.3.2 数据安全性

数据安全性是系统运行的基础，具体包括用户隐私保护，对敏感信息（如手机号）进行加密存储；文件校验，对上传的文件进行病毒扫描，防止恶意数据传播；权限控制，基于用户身份验证（如登录会话 ID）确保对数据访问的合法性；日志审计，记录所有关键操作，为异常行为的溯源提供支持。

第三章 系统设计

3.1 架构设计

3.1.1 设计思路

即时通讯系统作为高并发、实时性要求严苛的典型应用场景，其架构设计需要在性能、可扩展性、维护成本之间取得平衡。本系统的设计思路遵循“分而治之”的核心原则，将复杂的即时通讯功能拆解为多个职责单一、边界清晰的微服务。

设计目标主要包括：高可用性，通过服务冗余部署、故障自动转移机制，确保系统 7×24 小时不间断运行，可用性目标达到 99.9% 以上；高并发能力，单个服务实例能够处理 1000+ 并发连接，通过横向扩展支持万级用户同时在线；低延迟通信，端到端消息传输延迟控制在 200ms 以内，消息推送延迟不超过 50ms；可维护性，服务之间松耦合，单个服务的修改不影响其他服务，支持独立开发、测试、部署；可观测性，完善的日志、监控、告警体系，支持问题快速定位和性能瓶颈分析。

设计原则遵循：单一职责原则，每个服务只负责一个业务领域，如用户服务专注于用户管理，消息服务专注于消息传输与存储；无状态设计，服务本身不保存状态，状态信息存储到 Redis 等外部存储，支持服务的快速扩缩容；异步解耦，通过消息队列将同步调用转为异步处理，提升系统吞吐能力，降低服务间依赖；读写分离，数据库采用主从复制架构，写操作由主库处理，读操作分发到从库，提升数据库整体性能；缓存优先，热点数据优先从 Redis 缓存读取，缓存未命中时再访问数据库，降低数据库压力。

技术选型考量：后端语言选择 C++ 主要基于以下考虑：即时通讯系统对性能要求极高，C++ 作为系统级编程语言，在内存管理、并发模型、网络通信等方面具有天然优势。相比 Java、Go 等语言，C++ 在相同硬件条件下能够提供更高的吞吐量和更低的延迟。配合 brpc 框架，能够实现高效的 RPC 通信和 WebSocket 长连接处理。

3.1.2 微服务架构简介

本系统基于微服务架构进行设计和开发，将复杂的即时通讯功能拆分为多个独立的子服务，例如用户管理服务、好友管理服务、消息转发服务等。每个子服务聚焦于特定

领域，能够独立开发、测试、部署和扩展。这种模块化设计减少了服务之间的耦合，提高了系统的灵活性和可维护性。系统整体架构图如下：

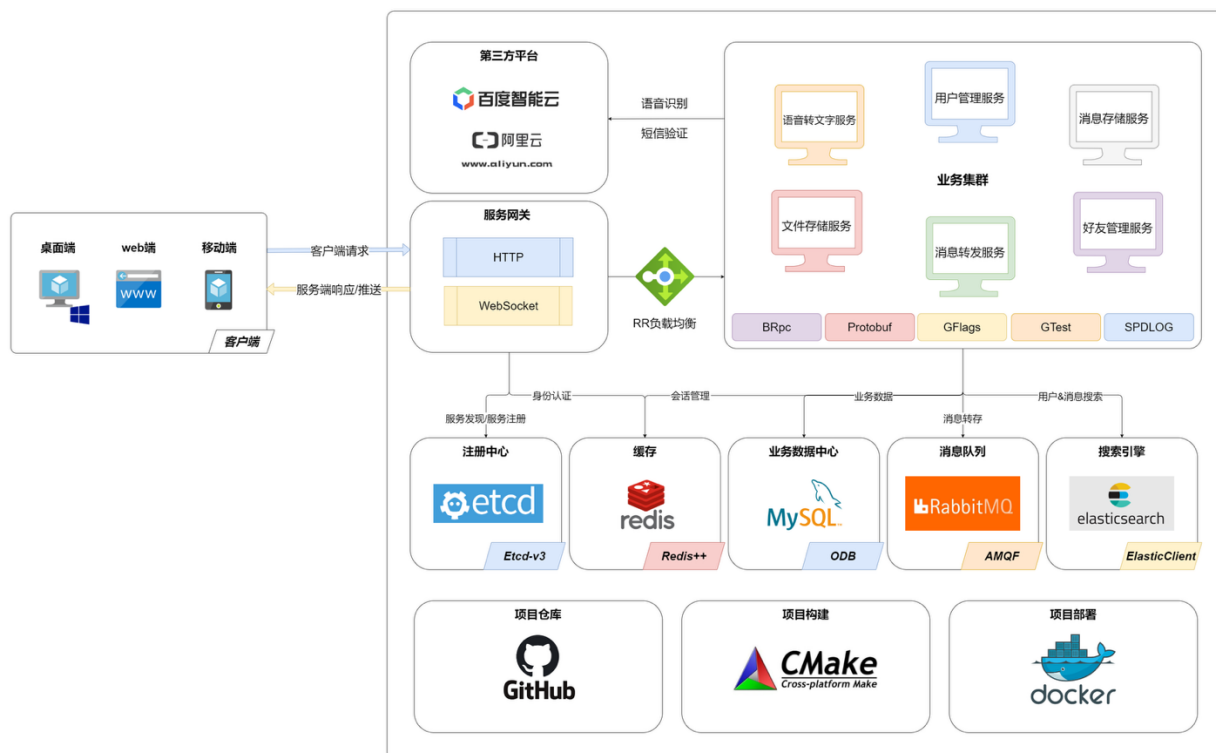


图 3-1_系统架构图

通过微服务架构的实现，系统在面对高并发场景时具备更好的弹性扩展能力。负载均衡、服务注册和发现等机制的引入，确保了高可用性。同时，不同的服务模块可以采用最适合其需求的技术栈和工具链，进一步提升开发效率。

3.1.2 数据库 ER 图

下面的 ER 图展示了用户、好友关系、聊天会话、消息记录等核心实体及其之间的关系。用户表与好友关系表通过用户 ID 关联，每个用户可以参与多个会话，会话中包含多个成员，消息记录表存储了与会话相关的所有消息。

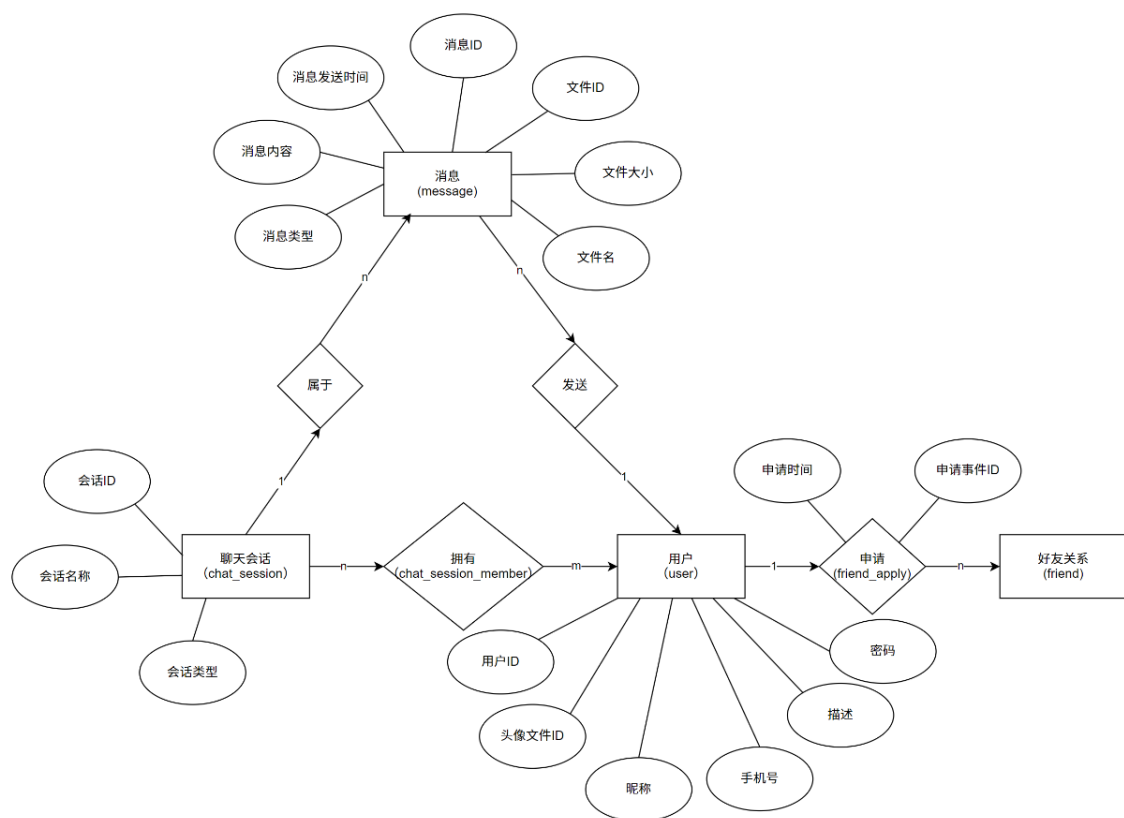


图 3-2_数据库 ER 图

3.2.2 数据库表设计

用户信息表 (user)：用户信息表用于存储系统中用户的基本信息，包括唯一标识符、昵称、签名等属性。这张表的数据支持用户注册、登录以及个人信息的管理功能。设置 user_id 为逻辑主键，通过唯一索引确保用户的唯一性；phone 字段用于手机号登录，支持快速查询；nickname 允许唯一性查询，避免昵称重复；对 description 和 avatar_id 设置非必须值，提供用户可选功能支持。表结构描述如下：

表 3-1 用户信息表 (user)

字段名	数据类型	键	索引类型	备注
id	bigint unsigned	是	PRI	自增主键，唯一标识
user_id	varchar(64)	否	UNI	用户唯一标识符
nickname	varchar(64)	否	UNI	用户昵称，支持唯一性
description	text	是	-	用户签名或描述
password	varchar(64)	否	-	用户密码，MD5 加密存储
phone	varchar(64)	否	UNI	用户手机号，支持唯一性
avatar_id	varchar(64)	是	-	用户头像文件标识

好友关系表 (relation): 好友关系表维护用户之间的好友关系，是一个双向的关系映射表。每当两人成为好友时，会在表中插入两条记录。表结构描述如下：

表 3-2 好友关系表 (relation)

字段名	数据类型	键	索引类型	备注
id	bigint unsigned	是	PRI	自增主键，唯一标识
user_id	varchar(64)	否	MUL	当前用户的唯一标识
peer_id	varchar(64)	否	MUL	好友的唯一标识

会话信息表 (chat_session): 会话信息表记录单聊和群聊会话的基本信息，支持多种会话类型。表结构描述如下：

表 3-3 会话信息表 (chat_session)

字段名	数据类型	键	索引类型	备注
id	bigint unsigned	是	PRI	自增主键，唯一标识
chat_session_id	varchar(64)	否	UNI	会话的唯一标识
chat_session_name	varchar(64)	否	-	会话名称（群聊名）

会话成员表 (chat_session_member): 会话成员表存储每个会话的成员列表信息，支持单聊和群聊会话。表结构描述：

表 3-4 会话成员表 (chat_session_member)

字段名	数据类型	主键	索引类型	备注
id	bigint unsigned	是	PRI	自增主键，唯一标识

消息记录表 (message): 消息记录表存储聊天会话的所有消息，支持文本消息和文件消息。表结构描述：

表 3-5 消息记录表 (message)

字段名	数据类型	主键	索引类型	备注
id	bigint unsigned	是	PRI	自增主键，唯一标识
message_id	varchar(64)	否	UNI	消息唯一标识符
session_id	varchar(64)	否	MUL	所属会话 ID
user_id	varchar(64)	否	-	消息发送者 ID
message_type	tinyint unsigned	否	-	消息类型（1-文本，2-文件）
create_time	timestamp	是	-	消息发送时间

字段名	数据类型	主键	索引类型	备注
content	text	是	-	文本消息的内容

3.2.3 数据库主从复制架构设计

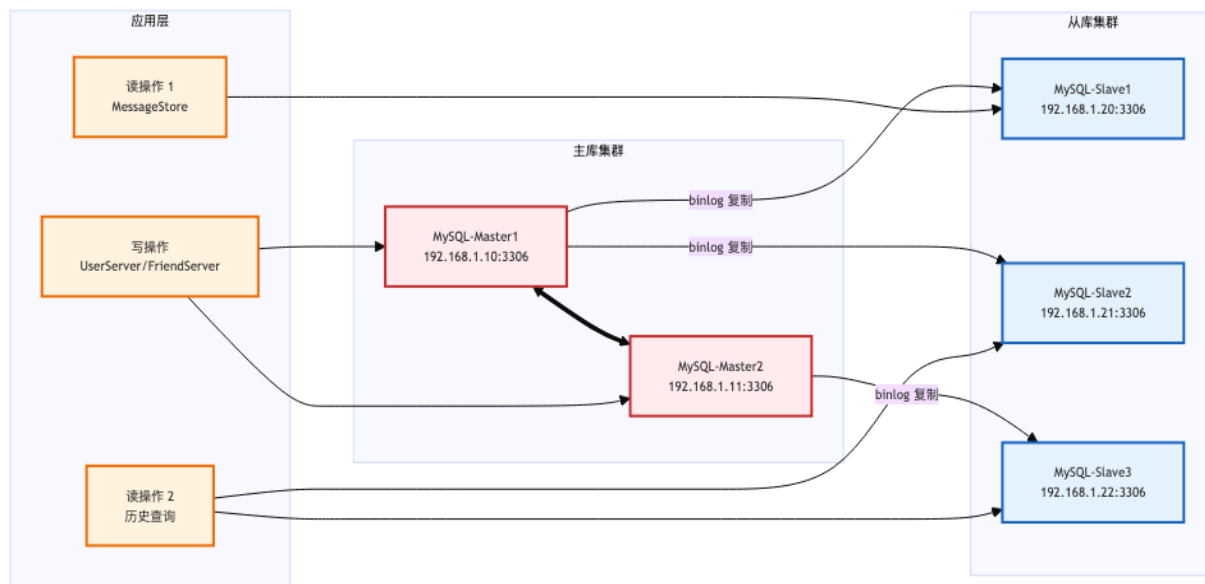


图 3-3 数据库主从复制架构

图 3-3 数据库主从复制架构展示了系统采用 MySQL 主从复制架构实现读写分离的设计。主库负责写入操作，从库承担读操作，通过读写分离和负载均衡，保障了数据库的高性能和高可用性。

为了提高数据库的读写性能和容错能力，系统采用主从复制架构。主库负责写入操作，从库承担读操作。通过读写分离和负载均衡，保障了数据库的高性能。

3.2.4 数据库索引优化策略

系统在设计数据库时充分利用了索引优化查询性能，首先，为用户表的 user_id、phone 等字段添加唯一索引，保证快速查询，接着考虑为好友关系表和会话成员表的外键字段设置普通索引，加速关联查询。消息表中 session_id 字段的索引支持快速检索会话内的消息记录。

3.2.5 数据库操作与事务管理

所有数据库操作均采用事务封装，确保数据一致性。以下是用户表的插入事务：

```
bool UserTable::insert(const std::shared_ptr<User> &user) {
    try {
        odb::mysql::transaction t(_db->begin());
        _db->persist(*user);
    }
}
```

```

        t.commit();
    } catch(const std::exception &e) {
        LOG_ERROR("向 mysql 新增用户数据失败! 原因: {}", e.what());
        return false;
    }
    return true;
}

```

类似的，其他表的操作均实现了插入、更新、查询和删除的事务管理，避免并发冲突和数据异常。

3.2.6 数据库索引策略总结

系统在数据库设计阶段充分考虑了查询性能优化的需求，为高频查询字段建立了合适的索引。如表 3-3 所示。

表 3-6 数据库索引策略表

表名	字段名	索引类型	索引名称	说明
user	user_id	唯一索引	idx_user_id	用户唯一标识
user	nickname	唯一索引	idx_nickname	昵称唯一性约束
user	phone	唯一索引	idx_phone	手机号唯一性约束
relation	user_id	普通索引	idx_user_id	好友关系查询
relation	peer_id	普通索引	idx_peer_id	反向关系查询
chat_session	chat_session_id	唯一索引	idx_session_id	会话唯一标识
chat_session_member	chat_session_id	普通索引	idx_session_id	会话成员查询
chat_session_member	user_id	普通索引	idx_user_id	用户参与会话查询
message	message_id	唯一索引	idx_message_id	消息唯一标识
message	session_id	普通索引	idx_session_id	会话消息查询
message	create_time	普通索引	idx_create_time	时间范围查询
friend_apply	user_id	普通索引	idx_user_id	申请人查询
friend_apply	peer_id	普通索引	idx_peer_id	被申请人查询

3.3 服务设计

3.3.1 用户管理服务设计

用户管理服务作为系统的核心模块之一，负责处理用户注册、登录、信息管理及相关的服务逻辑。在设计实现时，用户管理服务采用 C++ 编写，并结合 brpc 框架和多种数据存储技术（MySQL、Redis、Elasticsearch），以满足高并发环境下的性能需求。

用户管理服务的主要职责包括：用户注册，支持通过昵称 + 密码或手机号 + 验证码完成用户注册；用户登录，支持通过多种方式（例如昵称 + 密码、手机号 + 验证码）验证身份并生成会话；用户信息管理，包括用户头像、昵称、签名、手机号等信息的查询与更新；用户认证，基于会话管理实现用户登录状态的验证；数据存储与同步，将用户信息同步存储到 MySQL 和 Elasticsearch（ES）中。

用户管理服务作为微服务架构的一部分，支持高并发与扩展性，技术架构如下：MySQL 用于存储用户的基本信息，包括昵称、手机号、头像 ID 等；Redis 用于存储用户会话状态、登录标记及验证码信息，加速频繁访问；Elasticsearch（ES）用于存储用户的元信息，支持快速的用户搜索（例如通过昵称或手机号模糊匹配）；服务注册与发现通过 Etcd 进行，支持负载均衡与动态扩展；短信平台（DMS）集成第三方短信服务，用于手机号注册与登录的验证码发送。

用户注册流程如图 3-4 所示。

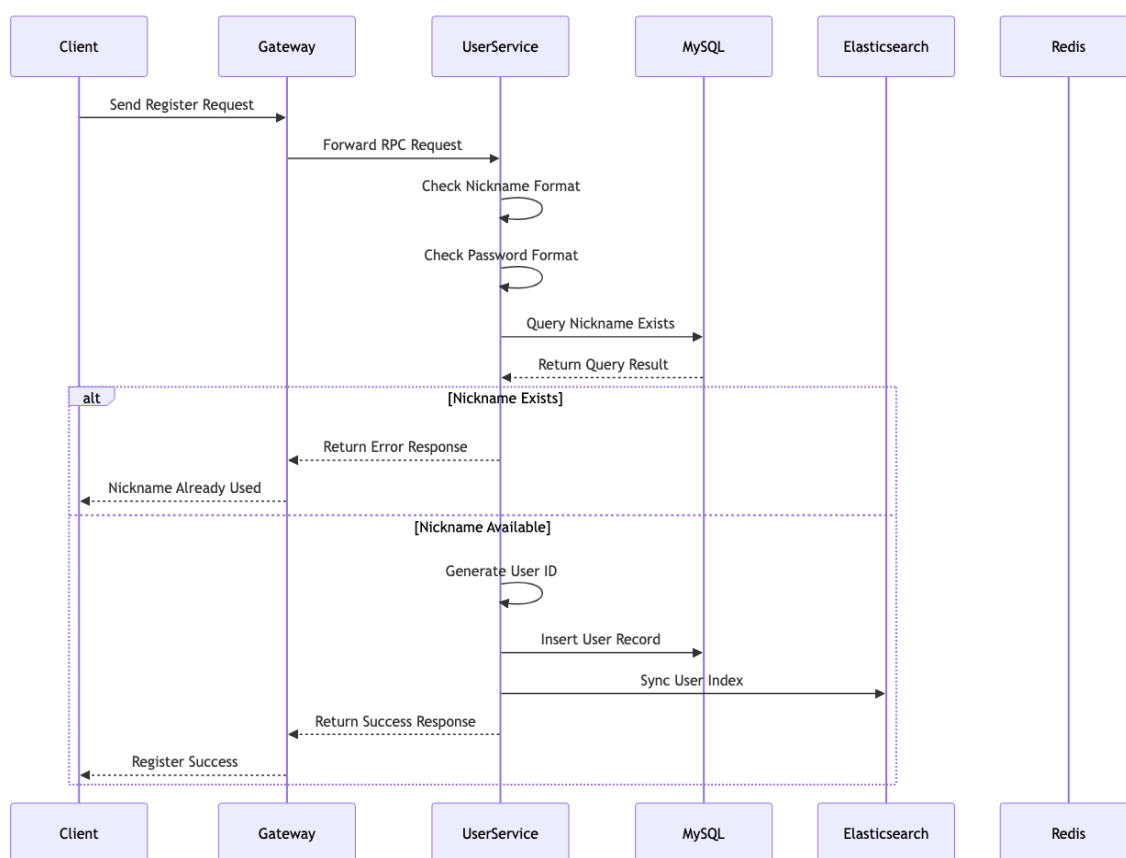


图 3-4 用户注册流程图

图 3-4 用户注册流程图展示了用户提交注册请求、服务校验昵称和密码格式、查询数据库、插入用户记录并同步到 Elasticsearch 的完整流程。

```
void UserServiceImpl::UserRegister(::google::protobuf::RpcController* controller, const ::chen_im::UserRegisterReq* request,
```

```

        ::chen_im::UserRegisterRsp* response,
        ::google::protobuf::Closure* done) {
    brpc::ClosureGuard rpc_guard(done);

    // 定义一个错误响应函数
    auto err_response = [response](const std::string& errmsg) {
        response->set_success(false);
        response->set_errmsg(errmsg);
    };

    // 第1步: 从请求中提取昵称和密码
    std::string nickname = request->nickname();
    std::string password = request->password();

    // 第2步: 检查昵称是否合法
    if (!nickname_check(nickname)) {
        LOG_ERROR("用户注册失败, 非法的昵称: {}", nickname);
        return err_response("非法的昵称格式! ");
    }

    // 第3步: 检查密码是否合法
    if (!password_check(password)) {
        LOG_ERROR("用户注册失败, 非法的密码: {}", password);
        return err_response("非法的密码格式! ");
    }

    // 第4步: 查询 MySQL, 检查昵称是否已存在
    auto existing_user = _mysql_user_table->select_by_nickname(nicknam
e);
    if (existing_user) {
        LOG_ERROR("用户注册失败, 昵称已被占用: {}", nickname);
        return err_response("昵称已被占用! ");
    }

    // 第5步: 生成唯一的用户 ID, 并插入 MySQL
    std::string user_id = generate_uuid();
    auto new_user = std::make_shared<User>(user_id, nickname, passwor
d);
    if (!_mysql_user_table->insert(new_user)) {
        LOG_ERROR("用户注册失败, MySQL 插入失败: {}", nickname);
        return err_response("数据库写入失败! ");
    }

    // 第6步: 同步用户信息到 Elasticsearch
    if (!_es_user->append_user(user_id, "", nickname, "", "")) {
        LOG_ERROR("用户注册失败, Elasticsearch 同步失败: {}", nickname);
        return err_response("搜索引擎数据同步失败! ");
    }

    // 第7步: 构建响应数据
    response->set_success(true);

```

```
response->set_user_id(user_id);  
LOG_INFO("用户注册成功: {}", nickname);  
}
```

在注册过程中，系统需要对昵称和密码进行校验。具体的校验逻辑如下：昵称校验要求长度限制在 3 至 15 个字符之间，仅允许字母、数字、下划线和连字符；密码校验要求长度限制在 6 至 15 个字符之间，仅允许字母、数字。以下是昵称和密码校验的辅助函数实现：

```
bool UserServiceImpl::nickname_check(const std::string& nickname) {  
    if (nickname.size() < 3 || nickname.size() > 15) {  
        return false;  
    }  
    for (char c : nickname) {  
        if (!isalnum(c) && c != '-' && c != '_') {  
            return false;  
        }  
    }  
    return true;  
}  
  
bool UserServiceImpl::password_check(const std::string& password) {  
    if (password.size() < 6 || password.size() > 15) {  
        return false;  
    }  
    for (char c : password) {  
        if (!isalnum(c)) {  
            return false;  
        }  
    }  
    return true;  
}
```

图 3-5 用户登录流程时序图展示了用户通过昵称 + 密码方式登录的完整流程，包括客户端请求、网关转发、用户服务验证、Redis 会话管理等关键步骤。

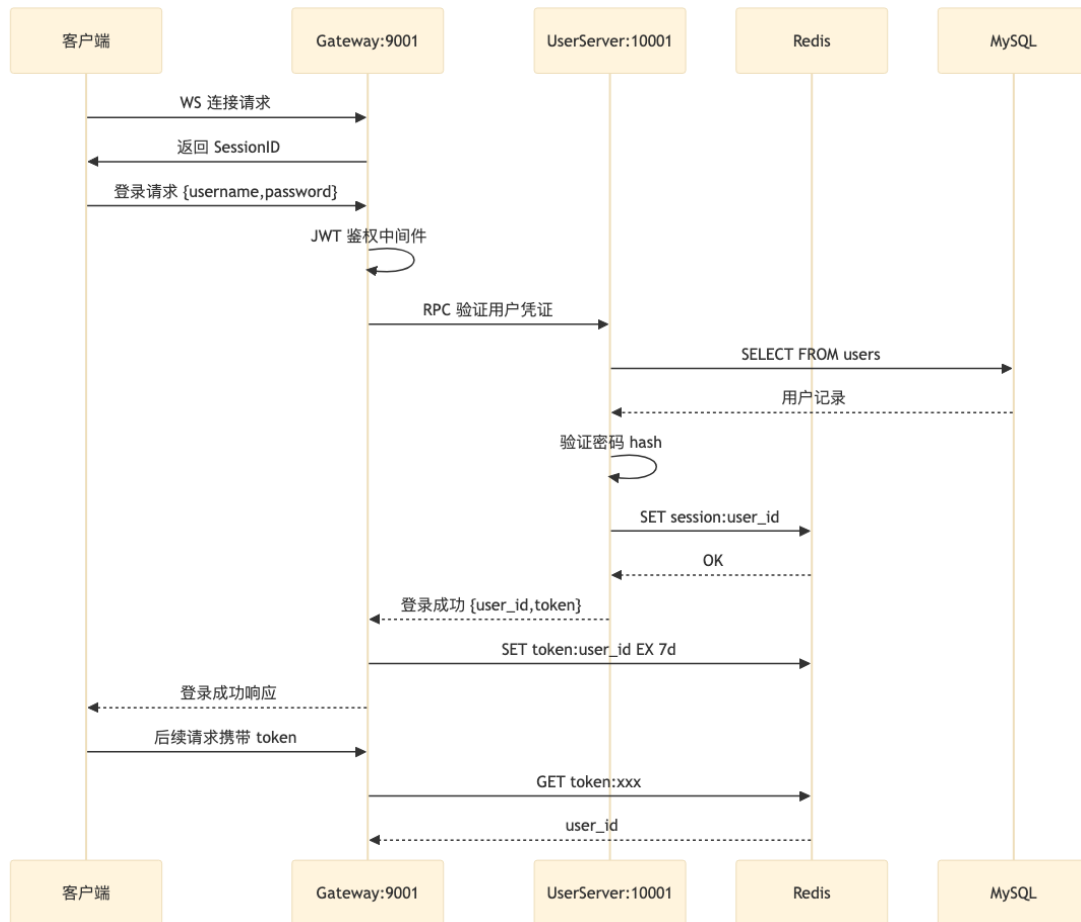


图 3-5 用户登录流程时序图

用户登录支持两种模式：昵称 + 密码登录，用户输入昵称和密码，验证后生成会话；手机号 + 验证码登录，用户输入手机号和短信验证码完成登录。

核心步骤如下：首先验证用户提交的数据格式；其次查询 MySQL 获取用户信息，验证密码或验证码；然后检查用户是否已登录（通过 Redis）；再生成会话 ID，存储到 Redis，标记用户登录状态；最后返回登录结果及会话 ID。

以下是昵称+密码登录的代码实现：

```

void UserServiceImpl::UserLogin(::google::protobuf::RpcController* controller,
                                const ::chen_im::UserLoginReq* request,
                                ::chen_im::UserLoginRsp* response,
                                ::google::protobuf::Closure* done) {
    brpc::ClosureGuard rpc_guard(done);

    // 定义错误响应
    auto err_response = [response](const std::string& errmsg) {
        response->set_success(false);
        response->set_errmsg(errmsg);
    };
}
    
```

```

// 提取请求参数
std::string nickname = request->nickname();
std::string password = request->password();

// 从 MySQL 查询用户信息
auto user = _mysql_user_table->select_by_nickname(nickname);
if (!user || user->password() != password) {
    LOG_ERROR("登录失败, 用户名或密码错误: {}", nickname);
    return err_response("用户名或密码错误! ");
}

// 检查 Redis 中的登录状态
if (_redis_status->exists(user->user_id())) {
    LOG_ERROR("登录失败, 用户已在其他地方登录: {}", nickname);
    return err_response("用户已在其他地方登录! ");
}

// 生成会话 ID, 存储登录状态
std::string session_id = generate_uuid();
_redis_session->append(session_id, user->user_id());
_redis_status->append(user->user_id());

// 返回登录成功
response->set_success(true);
response->set_session_id(session_id);
LOG_INFO("用户登录成功: {}", nickname);
}

```

3.3.2 好友管理服务设计

好友管理服务是整个系统的重要组成部分, 主要负责处理用户的好友关系维护、好友搜索、好友申请及管理用户聊天会话等功能。其设计目标是高效地管理用户关系, 并为用户提供清晰的好友管理接口, 同时通过与用户服务、消息服务的协作, 实现跨服务的数据整合。好友管理服务采用 C++ 编写, 基于 brpc 框架提供 RPC 服务接口, 底层依赖 MySQL 数据库存储好友关系和会话数据, 通过 Elasticsearch 实现用户搜索功能。

好友管理的核心功能涵盖七大模块。获取好友列表模块通过用户 ID 查询当前用户的所有好友, 并返回详细的好友信息, 包括昵称、头像、签名等完整资料。添加好友模块允许用户发起好友申请, 系统记录申请事件并通知被申请人, 申请事件采用 PENDING、ACCEPT、REJECT 三种状态机管理。处理好友申请模块支持被申请人接受或拒绝好友请求, 接受时系统自动更新好友关系并创建单聊会话。删除好友模块负责解除好友关系, 同时清理相关的会话数据, 包括从 relation 表删除好友关系记录、从 chat_session 表删除单聊会话、从 chat_session_member 表删除会话成员关系。搜索好友模块根据关键词(昵称、用户 ID 或手机号)搜索非好友用户, 搜索时通过

Elasticsearch 的布尔查询实现模糊匹配，并排除当前用户及其已有好友。获取聊天会话模块查询用户的历史聊天会话信息，包括单聊和群聊两种类型，系统从会话表和会话成员表中联合查询并组装完整会话信息。创建会话模块支持用户创建群聊会话，添加多个成员并记录会话信息，群聊会话的名称由创建者自定义。

(1) 添加好友流程

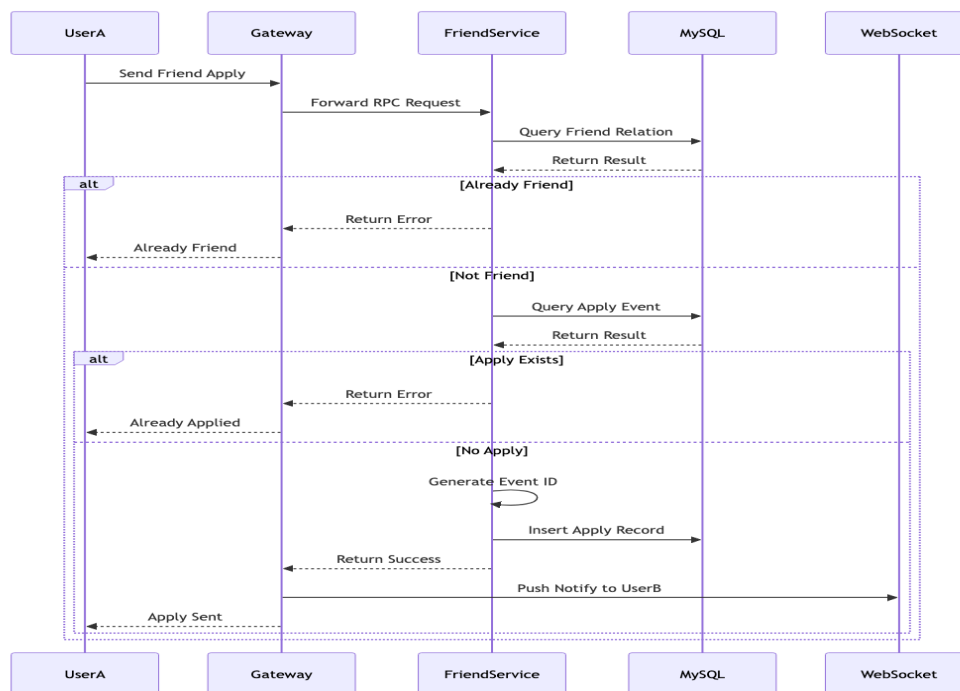


图 3-6 好友申请流程图

添加好友流程通过好友申请实现好友关系的添加，具体流程如下：

1. 用户发起好友申请，服务检查是否已有好友关系或未处理的申请事件。
2. 如果校验通过，将申请记录插入 friend_apply 表，并生成事件 ID。
3. 被申请用户接收申请后，选择接受或拒绝：
 - **接受：**更新 relation 表的好友关系，创建单聊会话并写入 chat_session 和 chat_session_member 表。
 - **拒绝：**删除申请事件记录，不更新好友关系。

以下是添加好友的核心代码逻辑：

```

void FriendServiceImpl::FriendAdd(::google::protobuf::RpcController* controller,
                                const ::chen_im::FriendAddReq* request,
                                ::chen_im::FriendAddRsp* response,
                                ::google::protobuf::Closure* done) {
    brpc::ClosureGuard rpc_guard(done);

    // 定义错误响应
    
```

```

auto err_response = [response](const std::string& errmsg) {
    response->set_success(false);
    response->set_errmsg(errmsg);
};

std::string request_id = request->request_id();
std::string uid = request->user_id(); // 申请人
std::string pid = request->respondent_id(); // 被申请人

// 检查两人是否已经是好友
if (_mysql_relation->exists(uid, pid)) {
    LOG_ERROR("{} - 已经是好友关系!", request_id);
    return err_response("已经是好友关系! ");
}

// 检查是否已有申请事件
if (_mysql_apply->exists(uid, pid)) {
    LOG_ERROR("{} - 已经申请过对方好友!", request_id);
    return err_response("已提交申请! ");
}

// 插入好友申请记录
std::string event_id = generate_uuid();
FriendApply apply(event_id, uid, pid);
if (!_mysql_apply->insert(apply)) {
    LOG_ERROR("{} - 插入好友申请事件失败!", request_id);
    return err_response("插入申请失败! ");
}

response->set_success(true);
response->set_notify_event_id(event_id);
}

```

(2) 获取好友列表流程

获取好友列表时，系统通过以下步骤实现：

1. 查询 relation 表获取用户的好友 ID 列表。
2. 调用用户服务批量获取好友详细信息。
3. 整理结果并返回给客户端。

```

void FriendServiceImpl::GetFriendList(::google::protobuf::RpcController
* controller,
                                     const ::chen_im::GetFriendListReq* request,
                                     ::chen_im::GetFriendListRsp* response,
                                     ::google::protobuf::Closure* done) {
    brpc::ClosureGuard rpc_guard(done);

    std::string request_id = request->request_id();
    std::string uid = request->user_id();
}

```

```

// 查询好友 ID 列表
auto friend_ids = _mysql_relation->get_friends(uid);
std::unordered_set<std::string> user_ids(friend_ids.begin(), friend_ids.end());

// 批量获取好友信息
std::unordered_map<std::string, UserInfo> user_info_map;
if (!_get_user_info(request_id, user_ids, &user_info_map)) {
    LOG_ERROR("{} - 获取好友信息失败!", request_id);
    return err_response("获取好友信息失败! ");
}

// 组织响应数据
response->set_success(true);
for (const auto& [id, info] : user_info_map) {
    response->add_friend_list()->CopyFrom(info);
}
}

```

3.3.3 消息存储服务设计

消息存储服务在系统中承担消息持久化、历史记录检索、关键字搜索等功能，是即时通讯系统的重要模块。消息存储服务通过 MySQL 实现消息元信息的可靠存储，通过 Elasticsearch 提供高效的关键字搜索，同时借助 RabbitMQ 实现消息的异步写入和分发。服务采用消费者模式从消息队列中异步消费消息，这种设计将消息转发与消息存储解耦，发送者无需等待存储完成即可收到确认，显著降低了端到端延迟。

消息存储服务的功能模块涵盖五个核心方面。消息写入模块通过 RabbitMQ 异步接收消息队列中的消息，并将消息写入 MySQL 和 Elasticsearch，对于文本消息直接存储内容，对于文件类消息先上传至文件服务获取文件 ID 后再存储元信息。历史消息查询模块支持用户通过会话 ID 和时间范围查询聊天会话中的历史消息，系统从 MySQL 中按时间排序检索消息元信息，再根据消息类型从 ES 或文件服务加载完整内容。最近消息查询模块用于用户登录后加载最近的聊天消息，系统从 MySQL 按时间倒序查询并限制返回数量，确保快速加载最新的聊天内容。消息搜索模块支持基于关键词的全文检索功能，利用 Elasticsearch 的 IK 分词器对文本消息内容进行分词和倒排索引，能够在海量消息中实现毫秒级检索响应。文件存储集成模块负责存储和加载消息中的文件内容，包括图片、语音、附件等多种类型，所有文件由文件服务统一存储管理，消息存储服务仅保存文件 ID 和元信息。

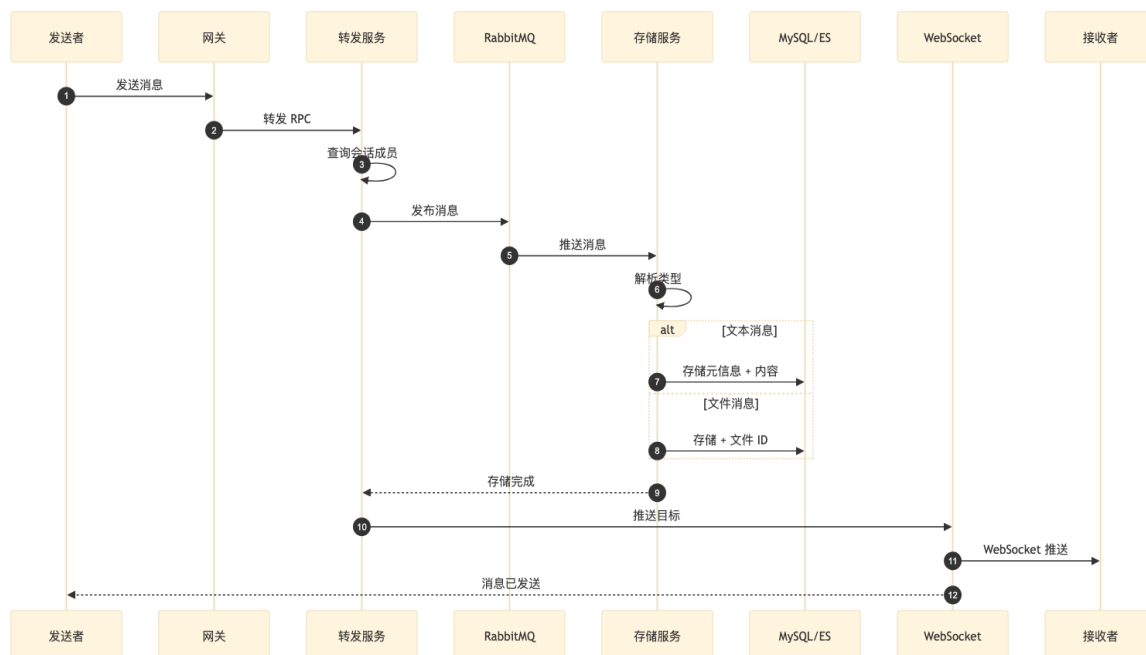


图 3-8 消息存储流程图

图 3-8 消息存储流程图展示了消息通过 RabbitMQ 异步写入、根据消息类型分别存储到 MySQL 和 Elasticsearch 的完整流程。

3.3.4 文件管理服务设计

文件管理服务在即时通讯系统中主要负责文件的存储和管理，支持文件的上传、下载、批量操作等功能。服务通过本地文件系统存储文件数据，并通过 brpc RPC 框架提供统一的文件管理接口。文件管理服务的設計遵循无状态原则，所有文件数据以唯一文件 ID 命名存储，支持多节点部署和负载均衡。

文件管理的核心功能涵盖四大模块。文件上传模块支持单个文件和多个文件的上传，将文件内容写入本地文件系统，并生成唯一的文件 ID 作为存储标识，文件 ID 采用 UUID 格式确保全局唯一性。文件下载模块支持通过文件 ID 获取单个文件或多个文件的内容，系统根据文件 ID 定位存储路径，读取文件数据后返回给请求者。批量操作模块支持批量文件的上传与下载操作，通过循环处理多个文件数据，减少网络往返次数，提升传输效率。服务注册与发现模块通过 etcd 注册中心实现文件服务的动态注册和节点管理，网关服务通过服务发现获取可用的文件服务节点列表，采用轮转策略进行负载均衡。文件上传流程如下：

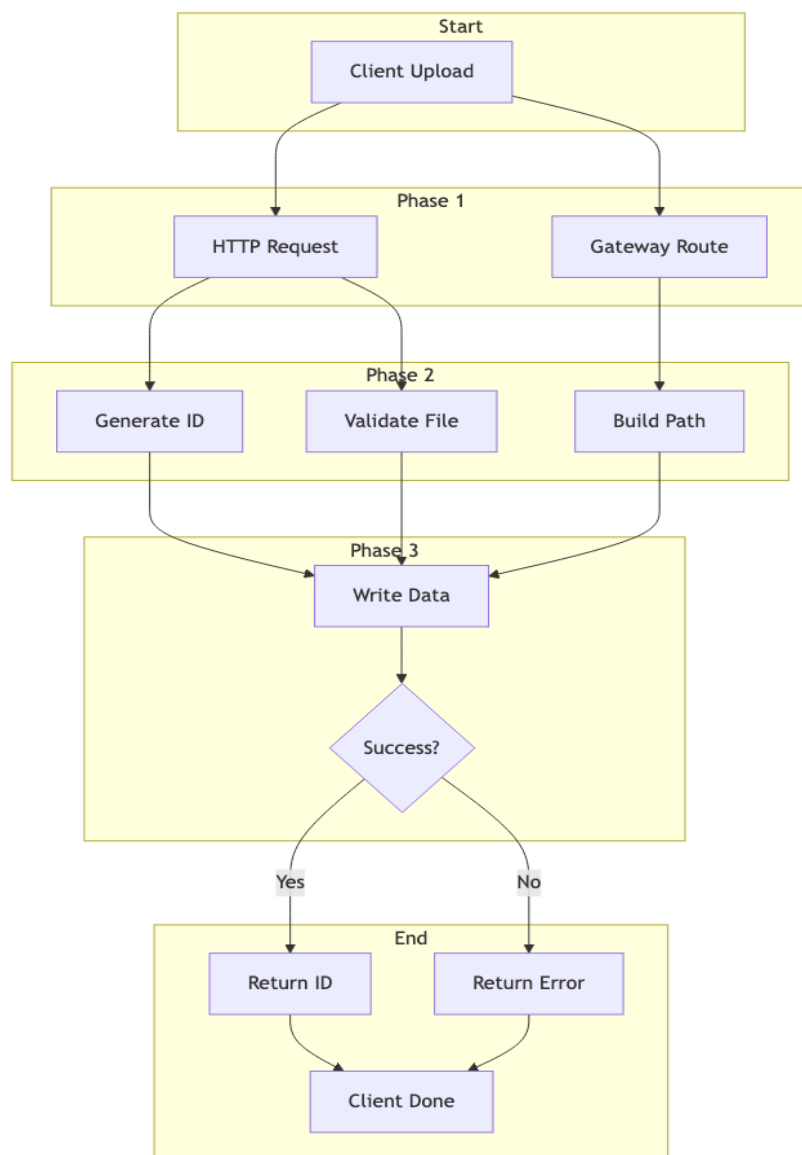


图 3-10 文件下载流程图

3.3.5 转发管理服务设计

转发管理服务的设计遵循”高效转发、异步解耦”的原则。作为消息传输的核心枢纽，转发服务需要在保证消息可靠性的同时，实现最低的传输延迟。

在消息转发方面，系统采用”推模式”设计。转发服务查询会话成员列表后，将目标用户 ID 列表返回给网关服务，由网关服务通过 WebSocket 连接主动推送给目标用户。这种设计避免了目标用户轮询查询消息，显著降低了服务器负载和消息延迟。

在消息持久化方面，系统采用异步写入设计。转发服务将消息发布到 RabbitMQ 消息队列后立即返回，无需等待存储完成。消息存储服务作为消费者异步消费消息，写入

MySQL 和 Elasticsearch。这种设计将消息转发与消息存储解耦，发送者几乎瞬间收到确认，显著提升了用户体验。

在服务依赖方面，转发服务依赖用户服务获取发送者的详细信息。这种依赖关系通过服务注册中心动态解析，转发服务无需硬编码用户服务的地址，支持服务的动态扩缩容和故障转移。

转发管理服务在即时通讯系统中主要负责将用户发送的消息转发给目标用户，同时将消息持久化存储到消息存储子服务中。服务通过 RabbitMQ 实现消息的可靠转发，并结合注册中心实现服务的动态注册与发现。转发管理服务的设计采用异步处理架构，将消息转发与消息存储解耦，发送者无需等待存储完成即可收到确认，显著降低了端到端延迟。

转发管理服务的核心功能涵盖四个方面。消息转发模块根据消息所属会话，将消息转发给会话中的所有成员，系统查询会话成员表获取目标用户 ID 列表，排除发送者自身后，将目标列表返回给网关服务用于 WebSocket 推送。消息持久化模块将消息发布到消息队列，供消息存储子服务进行持久化存储，消息采用 Protobuf 序列化后发布到 RabbitMQ 的指定交换机，确保消息格式的統一和高效传输。服务注册与发现模块通过 etcd 注册中心实现服务的动态注册和节点管理，支持横向扩展和高可用部署。依赖用户子服务模块从用户子服务获取消息发送者的详细信息，包括昵称、头像等，以便在消息推送时组装完整的消息结构。

(1) 消息转发流程

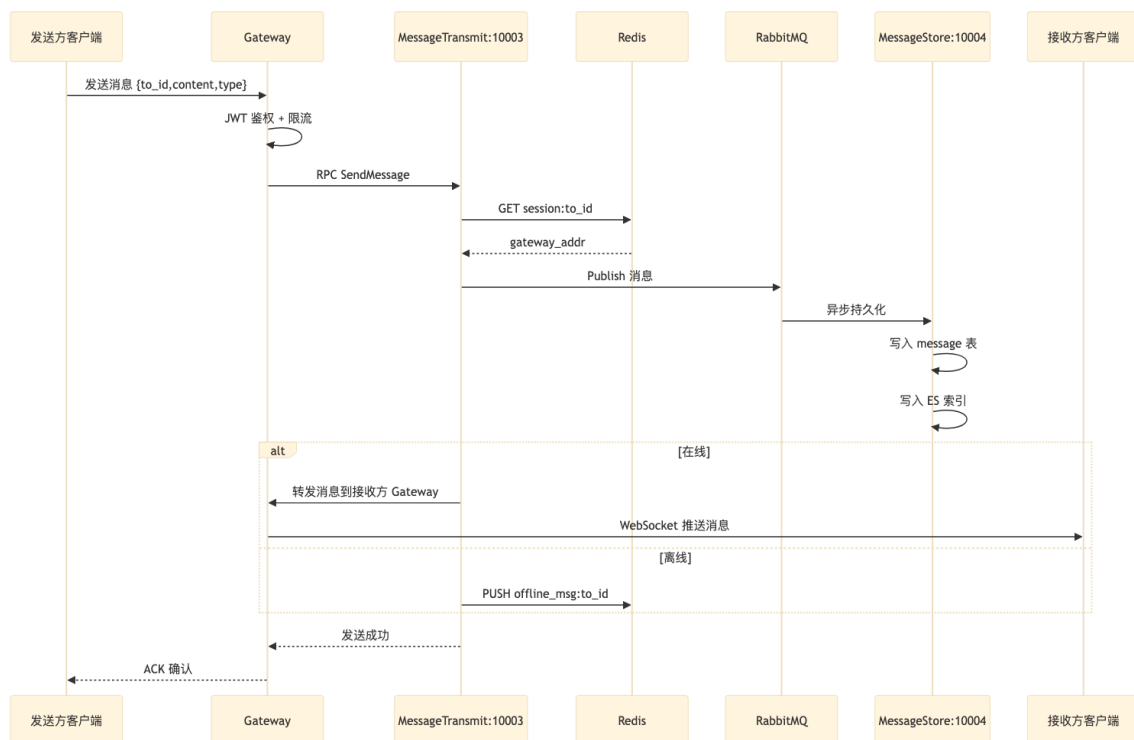


图 3-11 消息转发流程时序图

图 3-11 消息转发流程时序图展示了消息从客户端发送、网关转发、消息队列异步处理到最终推送给接收方的完整流程。

转发管理服务的核心流程是消息的转发，主要包括以下步骤：首先接收来自客户端的消息发送请求，提取消息的关键信息；其次调用用户子服务获取发送者的详细信息；然后查询数据库，获取消息所属会话的所有成员；再将消息封装后发布到 RabbitMQ 消息队列；最后返回消息转发成功的目标用户列表。

3.3.6 语音转文字服务

语音转文字服务是即时通讯系统的重要功能之一，主要用于将用户上传的语音数据转换为文本信息，方便后续的消息传递、存储以及搜索。该服务基于语音识别技术，利用第三方 SDK(如百度语音识别 SDK)进行语音处理,通过 brpc 框架提供高并发的 RPC 服务接口。

语音转文字服务的核心功能涵盖五大模块。语音识别模块通过语音识别 SDK，将用户上传的语音数据转换为可读文本，支持多种音频格式包括 WAV、MP3、AMR 等常见格式。多语言支持模块支持多语言识别，适应不同语言环境，包括普通话、粤语、英语等多种语言选项。错误处理与反馈模块处理语音识别中的错误情况，并将失败原因反馈给用户，常见错误包括音频格式不支持、音频质量过差、网络请求超时等。服务注册与动

态发现模块通过 etcd 注册中心实现服务的自动注册和动态发现，支持横向扩展和负载均衡。高性能 RPC 服务模块利用 brpc 提供高并发的 RPC 服务接口，结合线程池技术处理并发请求，确保在高并发场景下的响应性能。

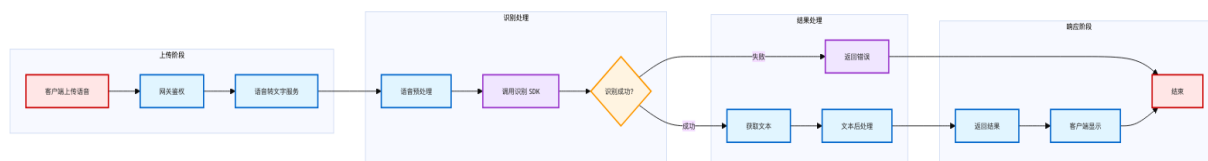


图 3-12 语音识别流程图

图 3-12 语音识别流程图展示了用户上传语音文件、服务调用第三方 SDK 进行识别并返回文字结果的流程。

语音识别流程如下：用户通过客户端上传语音文件，服务接收到语音数据后将其传递给语音识别 SDK 进行处理，SDK 返回识别结果，服务将识别结果打包并通过 RPC 响应返回给客户端。语音识别接口模块描述如下：

1. RPC 接口 SpeechRecognition 接收用户的语音数据，调用语音识别 SDK 获取识别结果。
 - a) 输入：语音数据 speech_content。
 - b) 输出：识别文本 recognition_result 或错误信息。
2. 基于第三方语音识别 SDK（如百度语音识别），实现语音数据的处理。
3. 利用 etcd 注册中心，注册语音识别服务，便于客户端动态发现。

3.4 系统部署设计

3.4.1 Docker 容器化部署

系统采用 Docker 容器化部署，将各微服务封装为独立的容器镜像。容器化部署的优势包括：环境一致性，开发、测试和生产环境保持一致，避免环境差异导致的问题；快速部署，通过镜像拉取和容器启动实现服务的快速部署；资源隔离，各服务运行在独立容器中，资源互不影响；易于扩展，通过增加容器实例实现服务的横向扩展。

3.4.2 Docker Compose 编排

使用 Docker Compose 实现多服务的统一编排与管理。docker-compose.yml 配置文件定义了所有基础设施服务的启动参数：

```

version: '3'
services:
  mysql:

```



```

image: mysql:8.0
environment:
  MYSQL_ROOT_PASSWORD: root123
  MYSQL_DATABASE: chen_im
ports:
  - "3306:3306"
volumes:
  - mysql_data:/var/lib/mysql

redis:
  image: redis:7.0
  ports:
    - "6379:6379"

etcd:
  image: bitnami/etcd:3.5
  environment:
    ETCD_ADVERTISE_CLIENT_URLS: http://etcd:2379
  ports:
    - "2379:2379"

rabbitmq:
  image: rabbitmq:3-management
  ports:
    - "5672:5672"
    - "15672:15672"

elasticsearch:
  image: elasticsearch:7.17
  environment:
    - discovery.type=single-node
  ports:
    - "9200:9200"

```

3.4.3 服务镜像构建

每个微服务通过 Dockerfile 构建独立的镜像。以用户服务为例：

```

FROM ubuntu:22.04
WORKDIR /app
COPY build/user_server /app/
COPY config.yaml /app/
EXPOSE 10001
CMD ["/user_server"]

```

服务构建流程：

1. 使用 CMake 编译 C++源码生成可执行文件。
2. 将可执行文件和配置文件复制到容器中。
3. 暴露服务端口并启动服务。

3.4.4 配置管理与日志系统

各服务通过配置文件加载运行参数，包括端口、数据库连接、超时设置等。配置文件采用 YAML 格式，便于解析和维护。

系统使用 SPDLOG 库进行日志记录，日志级别包括：DEBUG，调试信息，开发阶段使用；INFO，运行状态信息；WARN，警告信息，表示潜在问题；ERROR，错误信息，表示异常情况。日志输出到标准输出，通过 Docker 日志驱动收集和管理。

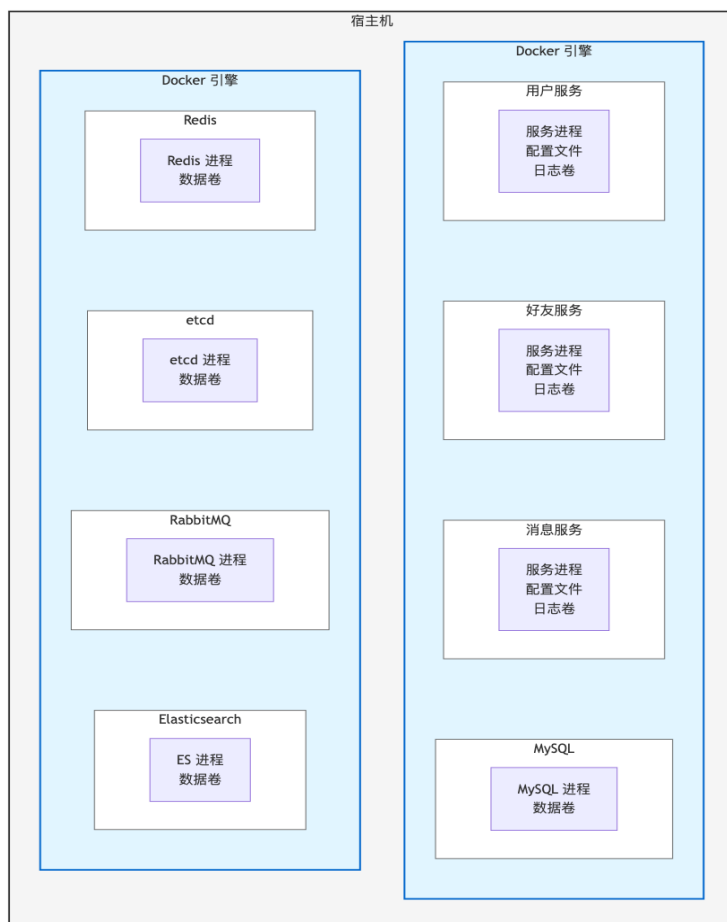


图 3-13 Docker 部署架构图

图 3-13 Docker 部署架构图展示了系统采用 Docker Compose 编排各微服务容器和基础设施组件的部署架构。

第四章 系统实现

4.1 前端架构概述

4.1.1 技术栈选型

前端采用 React 18 框架进行开发，技术栈包括：React 18 提供组件化开发模式和 Hooks 状态管理，支持虚拟 DOM 和函数式组件；Vite 作为新一代构建工具，基于 ES 模

块实现快速的冷启动和热更新，开发服务器启动时间小于 1 秒；Ant Design X 提供现代化的 UI 组件库，包含 Bubble、Sender、Prompts 等对话组件；Framer Motion 实现流畅的动画交互效果，支持手势识别和布局动画；Protobuf.js 用于前后端 Protobuf 协议通信，实现高效的二进制数据传输。

4.1.2 项目结构组织

前端项目采用模块化结构组织，源代码位于 src/目录下。项目结构按功能划分为四个层次：API 层负责与后端通信，组件层包含可复用的 UI 组件，上下文层管理全局状态，页面层包含路由级页面组件。

（1）API 层设计

API 层采用模块化设计，每个功能模块对应独立的 API 文件。httpClient.js 封装了 HTTP 请求和 Protobuf 编解码逻辑，实现了 encodeVarint、encodeStringField 等编码函数和 decodeVarint、decodeProtobufResponse 等解码函数。wsClient.js 封装了 WebSocket 连接管理和消息处理，支持断线重连机制。agentApi.js 封装了 Agent Server 的 HTTP 和 SSE 通信接口。各功能 API 文件(userApi.js、friendApi.js、messageApi.js) 提供类型化的接口函数。

（2）组件层设计

组件层按照功能划分为通用 UI 组件(components/ui/) 和业务组件。通用 UI 组件包括 Button、Input、Modal、Toast、Badge 等基础组件，采用 CSS 变量实现主题定制。业务组件包括 SessionList（会话列表）、MessageArea（消息区域）、FriendList（好友列表）、Sidebar（导航栏）、NotificationCenter（通知中心）等特定功能组件。agent 子目录包含 AI 助手相关组件：TaskSidebar（任务面板）、TaskTodoList（任务清单）、TaskThoughtChain（思维链）、ApprovalModalAntd（审批弹窗）等。

（3）状态管理层设计

状态管理采用 React Context 分层架构：AuthContext 管理用户认证状态，包括登录信息存储、Session 验证与自动恢复、登出状态清理；ChatContext 管理聊天消息和会话状态，包括会话列表、消息列表、好友列表、未读计数、通知中心等数据；AgentContext 管理 AI 助手任务状态，包括 Agent 用户列表、会话级 SSE 订阅、审批状态管理。各 Context 通过 useContext 钩子供子组件消费，支持状态共享和响应式更新。

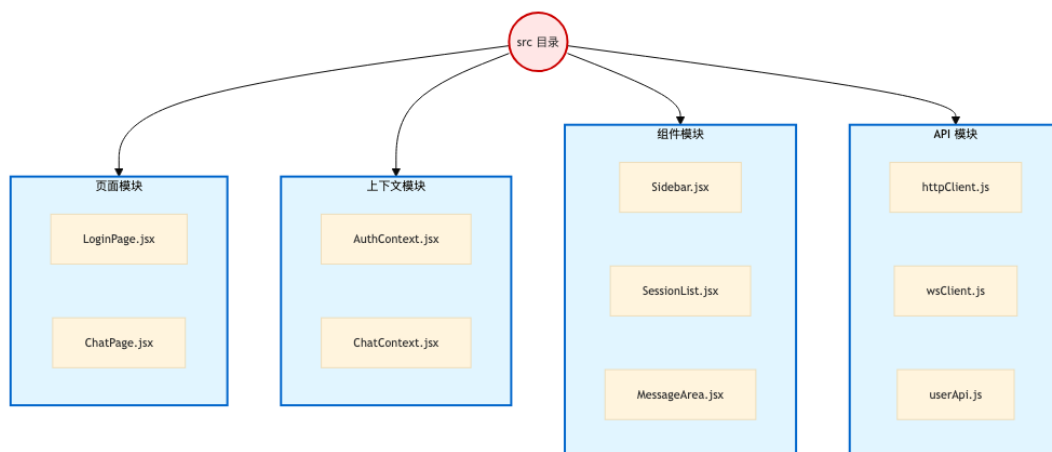


图 4-1：前端项目结构示意图

图 4-1 展示了前端项目的目录结构，包括 api、components、contexts、pages 四个主要模块的组织方式。

4.2 登录注册模块实现

4.2.1 登录页面界面设计

登录页面（LoginPage.jsx）是用户进入系统的入口，采用居中卡片式布局设计。页面背景使用渐变装饰圆形营造视觉层次感，主体表单卡片采用圆角设计配合阴影效果。页面顶部显示系统 Logo 图标，采用渐变背景的圆形设计。Logo 下方显示欢迎文字，根据当前模式显示”欢迎回来”或”创建新账户”。

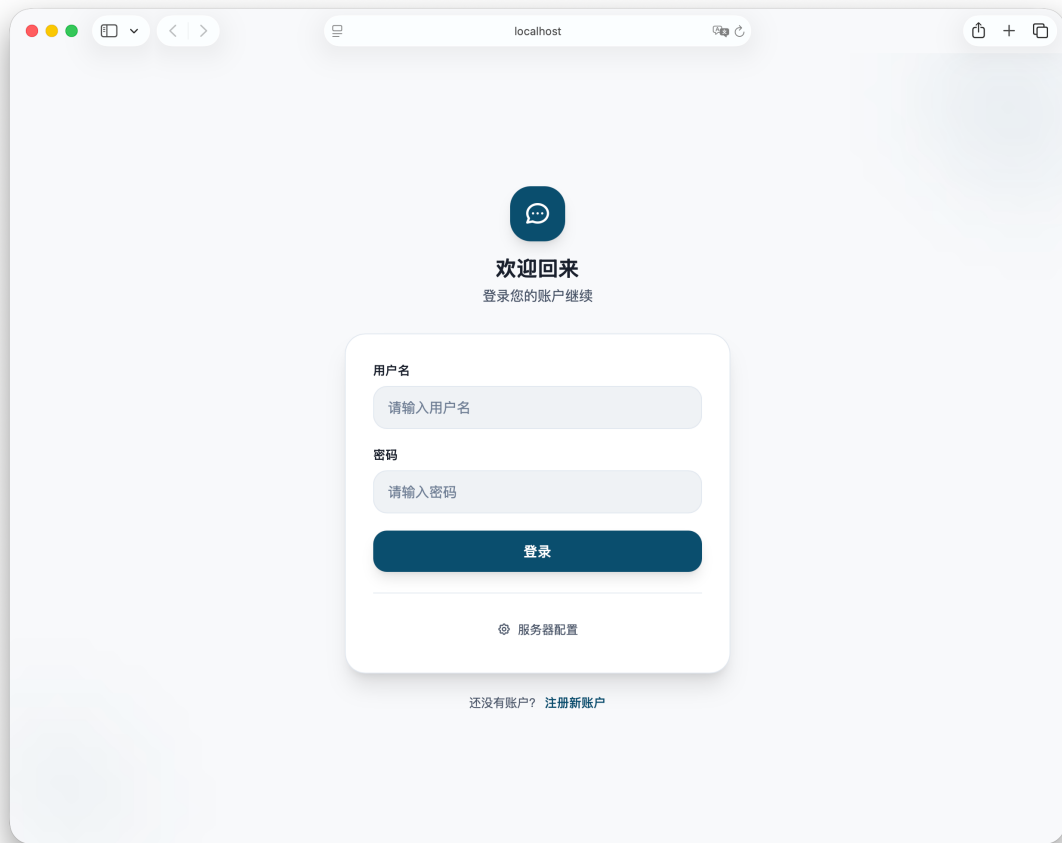


图 4-1：登录页面整体界面

图 4-1 展示了登录页面的完整界面，包括系统 Logo、欢迎文字、用户名输入框、密码输入框、登录按钮、注册切换链接和服务配置入口。

登录表单包含以下输入元素：用户名输入框，支持文本输入，显示占位符提示“请输入用户名”；密码输入框，类型为 password，显示占位符提示“请输入密码”；注册模式下额外显示确认密码输入框。表单下方是登录/注册按钮，采用渐变背景色，支持 hover 和 tap 动画效果。

4.2.2 登录表单交互逻辑

用户输入用户名和密码后点击登录按钮，触发 `handleSubmit` 函数。函数首先执行前端验证，检查输入是否为空。验证通过后，调用 `AuthContext` 提供的 `login` 函数发起登录请求。`login` 函数内部调用 `userApi.js` 中的 `usernameLogin` 接口，发送 Protobuf 格式的登录请求到后端 User Server。

登录请求的 Protobuf 编码过程如下：使用 `encodeUserLoginReq` 函数构造请求消息，包含 `request_id`、`nickname`、`password` 三个字段。字段编码采用 `varint` 格式存储标签

值，字符串字段采用 length-delimited 格式。编码后的二进制数据通过 fetch API 发送到网关服务。

登录成功后，后端返回 login_session_id。前端将其保存到 localStorage 的 AUTH_STORAGE_KEY 键下，同时调用 getUserInfo 接口获取用户详细信息。获取用户信息后，调用 wsClient.connect 函数建立 WebSocket 连接，用于接收实时消息推送。

登录过程中按钮显示加载状态，文字变为”登录中…“，防止用户重复提交。加载状态通过 useState 管理的 loading 变量控制，使用 disabled 属性禁用按钮交互。

```
const login = useCallback(async (nickname, password) => {
  const result = await usernameLogin(nickname, password);
  if (result.success) {
    const loginSessionId = result.login_session_id;
    setSessionId(loginSessionId);
    // 获取用户信息
    const userResult = await getUserInfo(loginSessionId);
    if (userResult.success && userResult.user_info) {
      setUser(userResult.user_info);
      // 保存到 localStorage
      localStorage.setItem(AUTH_STORAGE_KEY, JSON.stringify({
        sessionId: loginSessionId,
        user: userResult.user_info,
      }));
      // 连接 WebSocket
      wsClient.connect(loginSessionId);
      return { success: true };
    }
  }
  return { success: false, error: result.errmsg || '登录失败' };
}, []);
```

4.2.3 表单验证与错误处理

登录表单的前端验证包括以下规则：用户名和密码非空校验，输入为空时显示红色错误提示”请输入用户名和密码”；密码长度校验，注册模式下密码长度不少于 6 位时显示”密码长度至少 6 位”；密码一致性校验，注册模式下两次输入的密码不一致时显示”两次输入的密码不一致”。

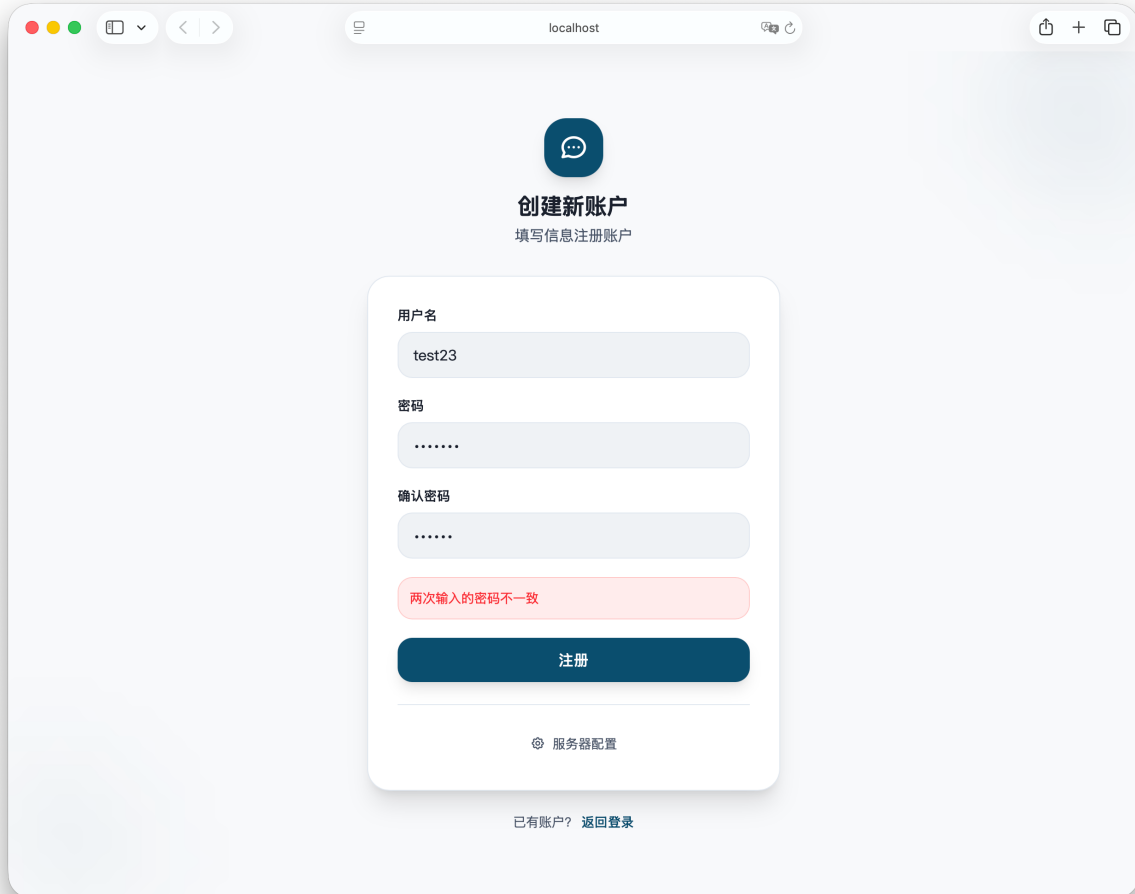


图 4-2：登录表单验证错误提示

图 4-2 展示了登录表单验证失败时的错误提示效果，红色背景的错误信息显示在表单顶部。

验证失败时，错误信息通过 `useState` 管理的 `error` 变量存储。错误提示组件使用 `motion.div` 实现淡入动画，背景色为半透明红色，边框为红色，文字为红色。成功提示使用类似的绿色样式。

后端返回的登录失败信息通过 `errmsg` 字段传递。前端解析 Protobuf 响应后，检查 `success` 字段是否为 `true`。如果 `success` 为 `false`，将 `errmsg` 内容显示为错误提示。

4.2.4 登录注册模式切换

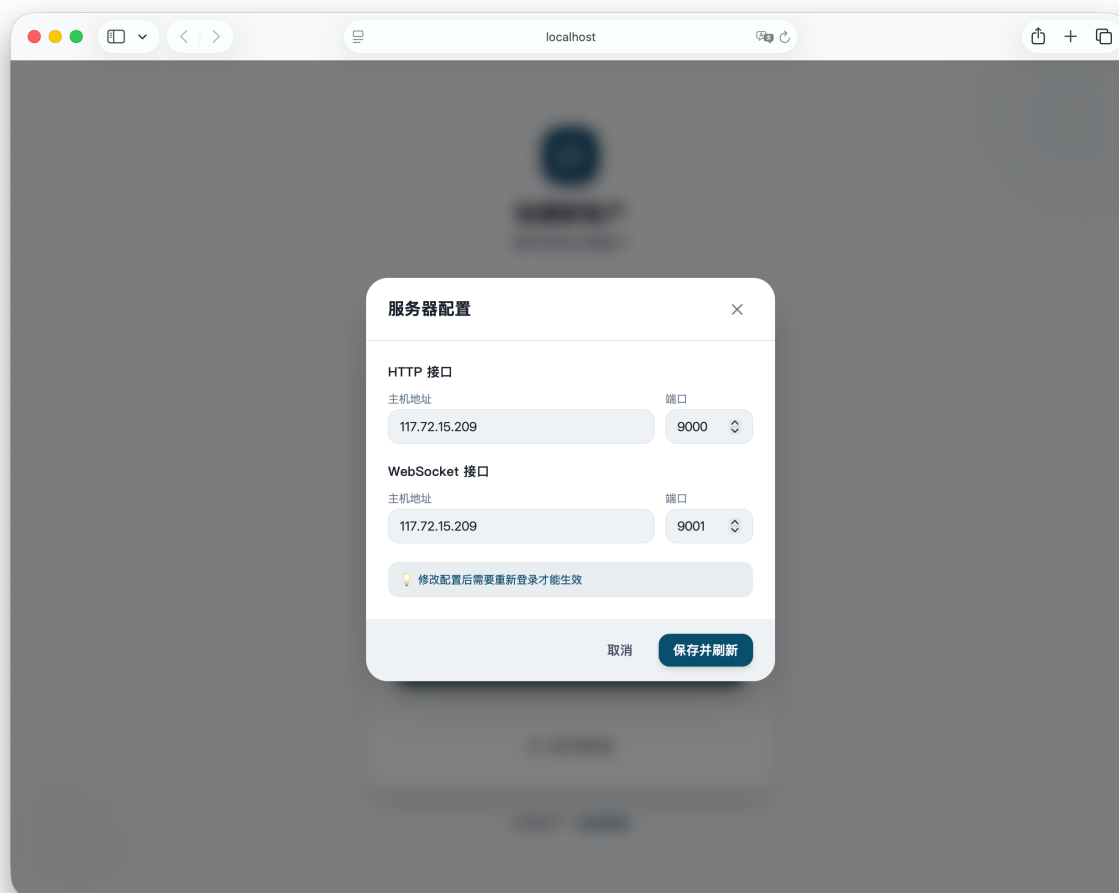
登录和注册模式通过 `isRegister` 状态变量切换，默认为登录模式。点击表单下方的“注册新账户”或“返回登录”链接触发 `toggleMode` 函数，切换 `isRegister` 状态并清空表单数据。

切换到注册模式时，确认密码输入框使用 Framer Motion 的 AnimatePresence 组件实现平滑展开动画。组件初始状态 opacity 为 0、height 为 0，展开后 opacity 变为 1、height 变为 auto。动画时长由 transition 参数控制，使用 spring 类型实现弹性效果。

```
{isRegister && (  
  <motion.div  
    initial={{ opacity: 0, height: 0 }}  
    animate={{ opacity: 1, height: 'auto' }}  
  >  
    <label className="block text-sm font-medium mb-2">  
      确认密码  
    </label>  
    <input type="password" value={confirmPassword}  
      onChange={(e) => setConfirmPassword(e.target.value)}  
      placeholder="请再次输入密码"  
      className="w-full px-4 py-3 rounded-xl border..."  
    />  
  </motion.div>  
)}
```

4.2.5 服务器配置功能

点击登录页面底部的“服务器配置”按钮，弹出 ServerConfig 模态框，允许用户修改后端服务器地址。模态框使用 fixed 定位覆盖全屏，背景为半透明黑色并添加模糊效果。模态框内容区域采用圆角白色背景，包含两个配置项：网关服务地址（HTTP API 入口）、WebSocket 服务地址（实时消息推送）。



此处插入图 4-3：服务器配置弹窗界面

图 4-3 展示了服务器配置弹窗的界面，包含网关地址、WebSocket 地址和 Agent 地址的输入框。配置信息保存到 localStorage，键名为 server_config。下次启动时，config.js 读取配置并构造请求 URL。修改配置后需重新登录才能生效。

4.3 聊天主页面布局实现

4.3.1 响应式三栏布局架构

聊天主页面（ChatPage.jsx）采用响应式三栏布局设计。左侧导航栏宽度固定为 80px，包含用户头像和功能切换按钮；中间栏宽度为 320px（移动端为 280px），显示会话列表或好友列表；右侧为主消息区域，占据剩余宽度。

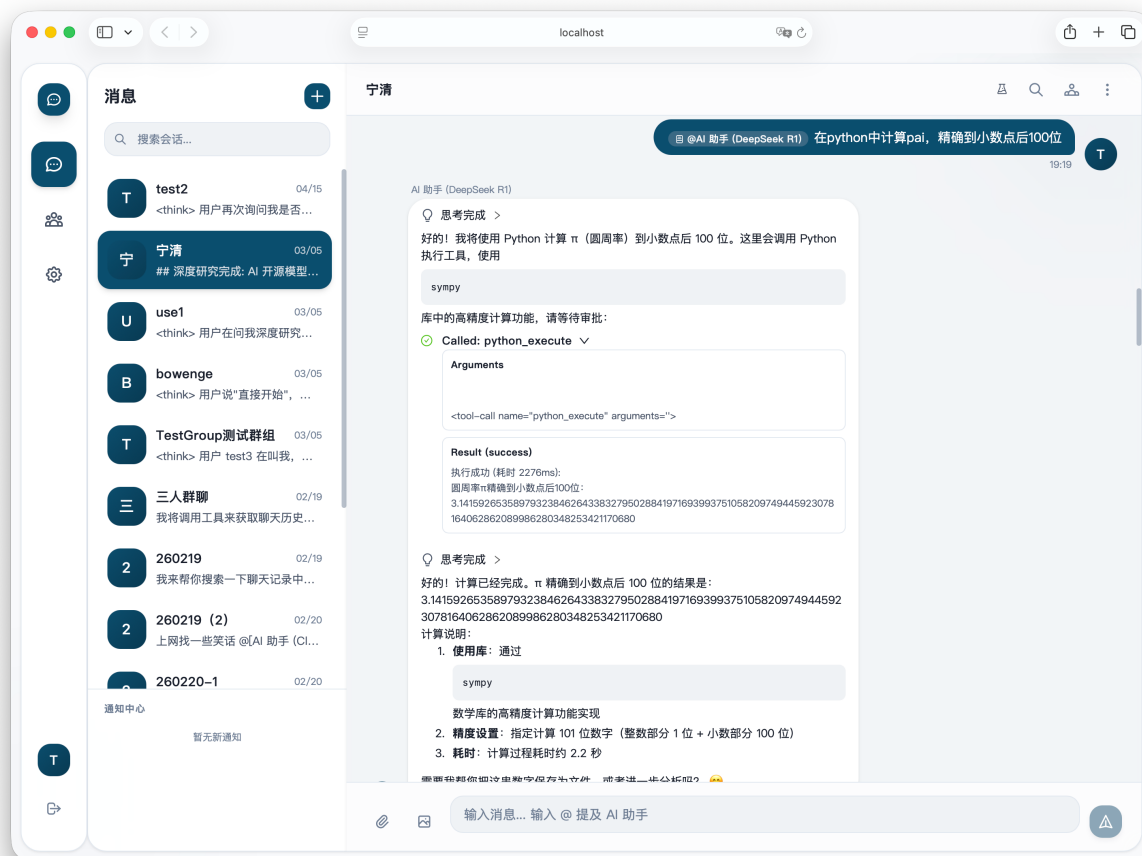


图 4-4：桌面端聊天主页面完整布局

图 4-4 展示了桌面端聊天主页面的完整三栏布局，左侧导航栏、中间会话列表、右侧消息区域。

布局结构通过 CSS Flexbox 实现，容器使用 `h-[100dvh] w-screen flex` 类名。左侧导航栏使用 `shrink-0` 类名固定宽度，中间栏使用 `flex-1 min-w-0` 类名弹性布局，右侧消息区域使用 `flex-1` 类名占据剩余空间。

4.3.2 桌面端布局实现

桌面端（屏幕宽度大于 1024px）完整展示三栏布局。左侧导航栏使用 `Sidebar` 组件渲染，宽度为 80px。导航栏包含四个区域：顶部 Logo 图标，使用渐变背景的圆角方形设计；导航按钮区域，包含消息、联系人、设置三个按钮；底部用户头像，点击弹出个人资料模态框；登出按钮，点击触发 `logout` 函数。

导航按钮使用 `motion.button` 组件渲染，支持 `hover` 缩放和 `tap` 反馈动画。当前激活的按钮使用 `primary` 背景色，其他按钮使用半透明背景。按钮图标使用 SVG 路径绘制，保证清晰度。

中间栏根据 activeTab 状态显示不同内容：activeTab 为 chat 时显示 SessionList 组件；activeTab 为 contacts 时显示 FriendList 组件；activeTab 为 settings 时隐藏中间栏，右侧全屏显示 SettingsPanel 组件。

右侧消息区域根据 activeTab 和 currentSession 状态显示不同内容：activeTab 为 chat 且 currentSession 存在时显示 MessageArea 组件；activeTab 为 contacts 时显示空状态占位图，提示用户选择联系人查看详情；currentSession 不存在时显示空状态占位图，提示用户选择会话开始聊天。

4.3.3 移动端布局适配

移动端（屏幕宽度小于 768px）采用单栏设计。左侧导航栏完全隐藏，使用 hidden md:flex 类名控制。底部显示固定导航栏，使用 fixed bottom-0 定位，高度为 64px（对应 h-16 类名）。底部导航栏包含三个按钮：消息、联系人、设置，与桌面端导航栏功能一致。



图 4-5：移动端聊天界面

图 4-5 展示了移动端聊天界面的单栏布局，底部导航栏切换消息、联系人和设置视图。

中间列表和右侧消息区域通过 showMobileChat 状态切换显示。初始状态 showMobileChat 为 false，显示中间列表（会话或好友）。用户点击某个会话后，触发 handleSessionSelect 函数，函数内部判断 window.innerWidth 小于 1024px 时设置 showMobileChat 为 true，切换到消息视图。

消息视图顶部显示返回按钮，点击触发 handleMobileBack 函数，设置 showMobileChat 为 false，返回列表视图。返回按钮使用 SVG 箭头图标，尺寸为 24px。

```
const handleSessionSelect = useCallback((sessionId) => {  
  selectSession(sessionId);  
  // 移动端自动切换到消息视图  
  if (window.innerWidth < 1024) {  
    setShowMobileChat(true);  
  }  
}, [selectSession]);
```

4.3.4 移动端手势交互

移动端手势交互通过触摸事件实现：左右滑动切换底部导航标签，使用 touch 事件监听手指移动距离；消息列表下拉刷新历史消息，使用 IntersectionObserver 监测滚动位置，触发 loadMessages 函数加载更多历史消息；长按消息弹出操作菜单，使用 touch 事件监听长按时间，超过 500ms 触发菜单显示。

4.4.4 会话选择交互

点击会话项触发 onClick 事件，调用 selectSession 函数切换当前会话。selectSession 函数执行以下操作：设置 currentSessionId 状态；清除该会话的未读计数；如果该会话没有加载历史消息，调用 loadMessages 函数获取历史消息。

会话项的激活状态通过 currentSessionId 判断。激活的会话项使用 primary 背景色，文字为白色，显示阴影效果。其他会话项使用透明背景，hover 时显示浅色背景。

4.4.5 创建群聊交互流程

点击头部右上角的加号按钮，弹出创建群聊模态框。模态框使用 fixed 定位覆盖全屏，背景为半透明黑色添加模糊效果。内容区域使用 motion.div 实现弹出动画，初始 scale 为 0.95，animate 后 scale 为 1。



图 4-8：创建群聊模态框

图 4-8 展示了创建群聊弹窗的界面，包含群名称输入框和好友成员选择列表。模态框包含以下表单元素：群名称输入框，使用 `useState` 管理的 `groupName` 变量存储内容，显示占位符“输入群聊名称...”；成员选择列表，从 `ChatContext` 获取 `friends` 数组，每个好友项显示头像、昵称和复选框。已选成员数量实时显示在列表标题中。

成员选择使用 `toggleMember` 函数处理。函数接收 `friendId` 参数，检查 `selectedMembers` 数组是否包含该 ID：如果包含则移除；如果不包含则添加。复选框使用 `accent` 类名设置主题色。

提交按钮点击触发 `handleCreateGroup` 函数。函数执行以下验证：群名称非空；至少选择一个成员。验证通过后，调用 `createChatSession` 接口，参数包括 `sessionId`、`userId`、`groupName` 和 `memberIdList`（包含当前用户 ID）。创建成功后刷新会话列表，关闭模态框，清空表单数据。

4.5 好友列表模块实现

4.5.1 好友列表组件结构

好友列表组件 (FriendList.jsx) 位于聊天页面中间栏，替代会话列表显示。组件结构包含三个部分：头部区域，显示标题”联系人”和搜索框；标签页区域，切换”好友”和”申请”两个标签；列表区域，根据 activeTab 状态显示不同内容。

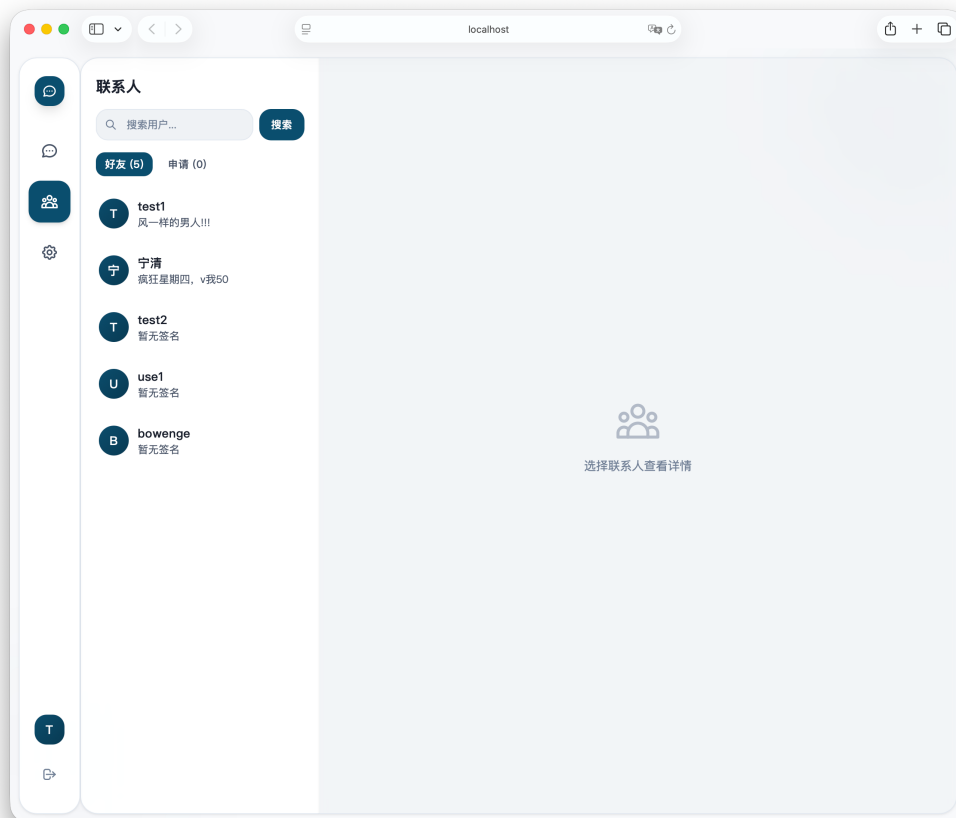


图 4-9：好友列表界面

图 4-9 展示了好友列表的完整界面，包括搜索框、好友/申请标签页和好友列表项。

头部区域使用 flex 布局，搜索框和搜索按钮水平排列。搜索框使用相对定位，左侧显示放大镜图标。搜索按钮使用 motion.button 组件，背景色为 primary，支持 hover 和 tap 动画。

标签页区域使用 flex 布局，两个按钮水平排列。当前激活的标签使用 primary 背景色，文字为白色；其他标签使用透明背景，文字为 secondary 色。标签文字显示数量，如”好友(5)”、“”申请(2)”。

4.5.2 搜索添加好友流程

输入搜索关键词后点击搜索按钮或按 Enter 键，触发 handleSearch 函数。函数首先检查 searchQuery 是否为空，然后调用 searchFriend 接口发送搜索请求。请求参数包括 sessionId、userId 和 searchKey（搜索关键词）。搜索好友的结果列表中出现每个结果项显示用户昵称和添加按钮。点击添加按钮触发 handleAddFriend 函数，调用

addFriendApply 接口发送好友申请。申请发送成功后弹出 alert 提示”好友申请已发送”。申请发送失败时显示 errmsg 内容。

4.5.3 好友申请处理流程

切换到”申请”标签页，显示待处理的好友申请列表。申请列表从 ChatContext 获取 friendRequests 数组，每个申请项 (FriendEvent) 包含 event_id 和 sender 字段 (UserInfo 类型)。

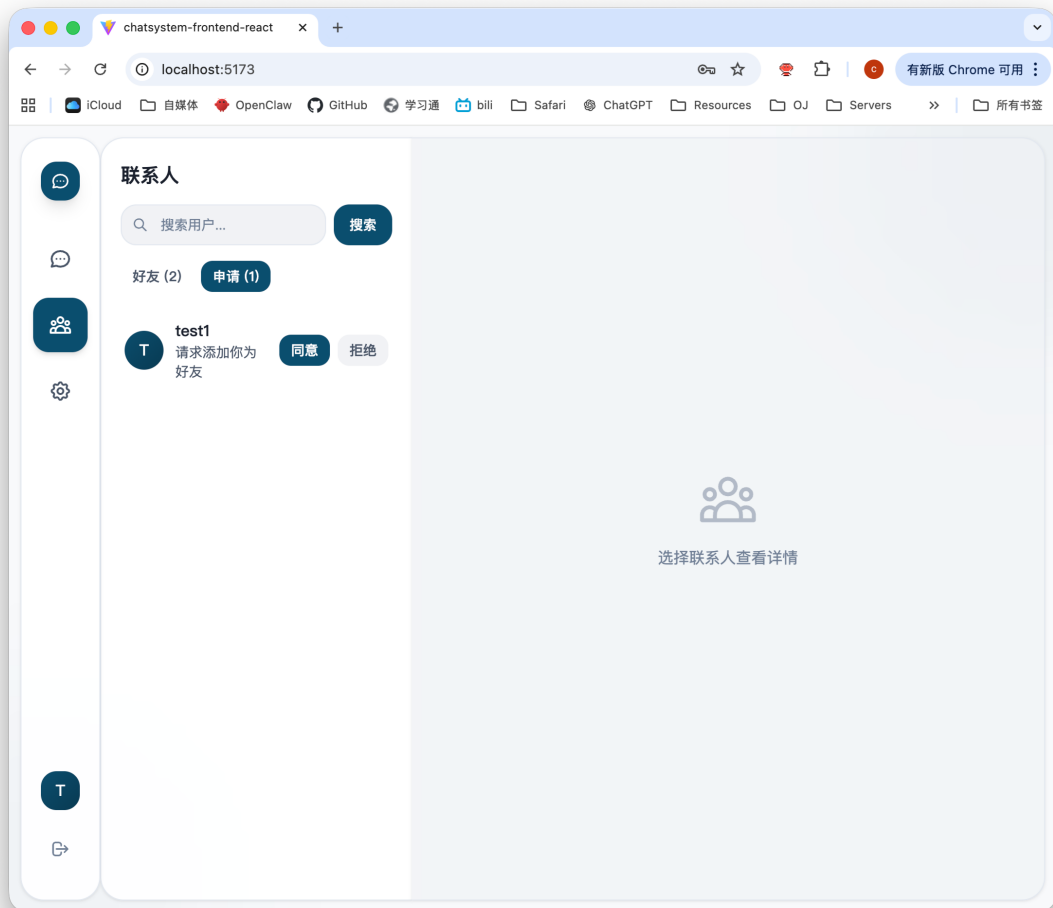


图 4-11：好友申请列表界面

图 4-11 展示了好友申请列表，每个申请项显示申请人信息和同意/拒绝按钮。

申请项使用 motion.div 组件渲染，支持渐入动画。内容包含：申请人头像，使用 Avatar 组件渲染；申请人昵称；操作按钮区域，包含”同意”和”拒绝”两个按钮。

点击同意按钮触发 handleFriendRequest 函数，参数包括 applyUserId、eventId 和 agree=true。函数调用 addFriendProcess 接口处理申请。处理成功后刷新好友列表和申请列表。

点击拒绝按钮触发类似流程，参数 `agree=false`。处理成功后仅刷新申请列表，好友列表不变。

```
const handleFriendRequest = async (applyUserId, eventId, agree) => {  
  if (!userId) return;  
  const result = await addFriendProcess(sessionId, userId, applyUserId, agree, eventId);  
  if (result.success) {  
    loadFriendRequests();  
    if (agree) {  
      loadFriends();  
    }  
  } else {  
    alert(result.errmsg || '处理失败');  
  }  
};
```

4.6 消息区域模块实现

4.6.1 消息区域组件结构

消息区域组件（`MessageArea.jsx`）是系统的核心组件，负责展示聊天消息、发送新消息和管理会话。组件结构包含五个部分：桌面端头部，显示会话名称和功能按钮；移动端头部，显示会话名称和返回按钮；消息列表区域，使用 `flex` 布局占满剩余空间；消息输入组件（`MessageInput`），固定在底部；搜索栏，点击搜索按钮后展开显示。

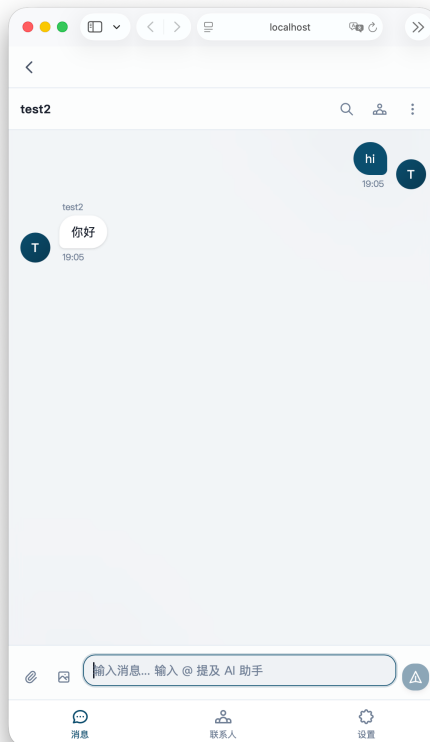


图 4-12：消息区域聊天界面

图 4-12 展示了消息区域的完整界面，包括会话名称、消息气泡列表和底部输入框。

桌面端头部高度为 64px（对应 h-16 类名），使用 flex 布局水平排列。左侧显示会话名称，使用 truncate 类名限制最大宽度 200px。右侧显示三个功能按钮：搜索按钮，点击展开搜索栏；成员管理按钮，点击弹出成员模态框；更多按钮，点击弹出会话信息模态框。

4.6.2 消息气泡布局设计

消息列表使用 flex 布局，支持自动滚动到底部。消息采用气泡式布局设计：用户自己发送的消息靠右显示，使用 flex-row-reverse 类名实现；他人发送的消息靠左显示，使用默认 flex-row 类名。

每个消息项包含三个元素：头像，使用 Avatar 组件渲染，尺寸为 md（40px），点击触发 handleAvatarClick 函数显示用户信息卡片；消息气泡，根据消息类型和发送者使用不同样式；时间戳，显示在气泡下方，格式为时分。

消息气泡样式根据发送者和消息类型决定：用户消息使用 bubble-self 背景色（蓝色渐变），文字为白色，圆角为 rounded-2xl rounded-br-md（右下角小圆角）；他人消息使用 bubble-other 背景色（白色/灰色），文字为深色，圆角为 rounded-2xl rounded-bl-md（左下角小圆角）；图片消息不使用气泡背景，直接显示图片。

4.6.3 消息类型渲染实现

系统支持四种消息类型的渲染，通过 renderMessageContent 函数处理：

（1）文本消息渲染

文本消息（message_type=0）直接显示 content 字段内容。如果发送者是 AI Agent（user_id 以 "agent" 开头），使用 StreamingMarkdown 组件渲染，支持 Markdown 语法和流式输出。普通用户消息使用 p 标签渲染，调用 renderTextWithAgentMentions 函数处理 @提及标记，将 @[昵称]{agent-id} 格式替换为高亮显示的昵称文本。

（2）图片消息渲染

图片消息（message_type=1）使用 LazyImageMessage 组件渲染。组件采用懒加载策略，初始显示占位图，当图片元素进入视口时才加载实际内容。加载过程通过 IntersectionObserver 监测，进入视口后调用 fetchImage 函数获取 Base64 数据。加载完成后显示实际图片，点击图片弹出全屏预览模态框。

（3）文件消息渲染

文件消息 (message_type=2) 使用卡片式设计。卡片包含：文件图标，根据扩展名显示不同颜色和标签（如 PDF 显示红色 PDF 标签）；文件名称，使用 truncate 类名截断过长文本；文件大小，调用 formatFileSize 函数格式化；下载按钮，点击触发文件下载。

(4) 语音消息渲染

语音消息 (message_type=3) 使用简约设计，显示麦克风图标和”[语音消息]“文字标识。

4.6.4 消息发送交互流程

发送文本消息的交互流程通过 handleSendMessage 函数实现。函数执行以下步骤：

第一步，创建本地消息对象。消息对象包含：message_id 使用 local_前缀和当前时间戳构造；chat_session_id 为当前会话 ID；timestamp 为当前时间；sender 为当前用户信息；message 为消息内容，message_type 为 0，string_message 包含 content 字段；_pending 标记为 true，表示消息正在发送。

第二步，乐观更新界面。调用 ChatContext 提供的 addMessage 函数，将本地消息添加到消息列表。消息立即显示在界面上，_pending 标记使消息下方显示”发送中…“提示。

第三步，发送网络请求。调用 sendTextMessage 接口发送消息。接口使用 Protobuf 编码请求消息，包含 request_id、user_id、session_id、chat_session_id 和 message 字段。

第四步，处理发送结果。发送成功后，延迟 3 秒调用 updateMessageStatus 函数，移除 _pending 标记。发送失败时，消息保留 _pending 标记，显示错误提示。

```
const handleSendMessage = async (content) => {
  if (!content.trim() || !currentSession) return;
  // 创建本地消息
  const localMessage = {
    message_id: `local_${Date.now()}`,
    chat_session_id: currentSession.chat_session_id,
    timestamp: Date.now(),
    sender: { user_id: user.user_id, nickname: user.nickname, avatar: user.avatar },
    message: { message_type: 0, string_message: { content } },
    _pending: true,
  };
  // 乐观更新
  addMessage(currentSession.chat_session_id, localMessage);
  // 发送请求
  try {
```

```

        await sendTextMessage(sessionId, user.user_id, currentSession.ch
at_session_id, content);
    } catch (error) {
        console.error('发送消息失败:', error);
    }
};

```

4.6.5 图片文件发送流程

发送图片和文件的流程类似,通过 `handleSendImage` 和 `handleSendFile` 函数实现。

发送图片流程: 用户点击图片按钮, 弹出系统文件选择器, 限制文件类型为 `image/*`。选择文件后, 使用 `FileReader` 读取文件内容为 Base64 格式。创建本地消息, `message_type` 为 1, `image_message` 包含 `image_content` 字段。调用 `sendImageMessage` 接口发送请求, 请求编码时将 Base64 数据转换为 `Uint8Array`。

发送文件流程: 用户点击附件按钮, 弹出系统文件选择器, 无类型限制。选择文件后, 使用 `FileReader` 读取文件内容为 Base64 格式。创建本地消息, `message_type` 为 2, `file_message` 包含 `file_name` 和 `file_size` 字段。调用 `sendFileMessage` 接口发送请求, 请求编码时包含完整的文件数据。

文件大小限制: 图片文件不超过 5MB, 普通文件不超过 50MB。超出限制时弹出 `alert` 警告。

图 4-13: 消息发送中的 pending 状态

4.6.6 消息搜索功能实现

消息区域顶部提供搜索按钮, 点击后展开搜索栏。搜索栏使用 `absolute` 定位, 覆盖在消息列表上方。搜索栏包含: 搜索输入框, 使用 `useState` 管理的 `searchQuery` 变量存储关键词; 搜索按钮, 点击触发 `handleSearch` 函数; 取消按钮, 点击关闭搜索栏并清空结果。

搜索流程: 输入关键词后点击搜索按钮或按 `Enter` 键, 调用 `searchMessages` 接口。接口参数包括 `sessionId`、`userId`、`chatSessionId` 和 `searchKey`。搜索结果存储在 `searchResults` 数组中, 以下拉列表形式展示。每个结果项显示发送者昵称和消息内容预览。

4.7 消息输入模块实现

4.7.1 消息输入组件结构

消息输入组件 (MessageInput.jsx) 位于消息区域底部, 高度自适应内容。组件结构包含: 隐藏的文件选择器, 使用 `input type=“file”` 实现; 输入框容器, 使用相对定位支持@提及下拉框; 多行文本域, 支持自动增高; 附件按钮, 触发文件选择; 图片按钮, 触发图片选择; 发送按钮, 提交消息。输入框使用 `textarea` 元素渲染, `rows` 属性为 1, `maxHeight` 限制为 120px。placeholder 根据 `agentMembers` 数组长度显示不同提示: 有 Agent 时显示”输入消息… 输入@提及 AI 助手“; 无 Agent 时显示”输入消息… “。

4.7.2 键盘快捷发送实现

输入框支持键盘快捷发送: 按 Enter 键直接发送消息; 按 Shift+Enter 组合键插入换行。这一行为通过 `handleKeyDown` 函数实现, 监听 `onKeyDown` 事件。

事件处理逻辑: 首先检查 `mentionState.visible` 状态, 如果@提及下拉框正在显示, 不处理键盘事件, 让下拉框处理上下键选择。其次检查 `e.key` 是否为 Enter 且 `e.shiftKey` 为 false, 满足条件时调用 `preventDefault` 阻止默认换行行为, 然后调用 `handleSubmit` 发送消息。

```
const handleKeyDown = (e) => {
  if (mentionState.visible) return;
  if (e.key === 'Enter' && !e.shiftKey) {
    e.preventDefault();
    handleSubmit(e);
  }
};
```

4.7.3 @提及功能实现

输入框支持@提及 AI 助手的功能。用户输入@字符后, 弹出下拉列表展示可用的 Agent 用户。选择 Agent 后, 输入框插入格式为@[昵称] {agent-id} 的提及标记。



图 4-15: @提及下拉列表界面

图 4-15 展示了输入@后弹出的 Agent 选择下拉列表。

@触发检测通过 `handleChange` 函数实现, 监听 `onChange` 事件。函数执行以下逻辑: 获取当前输入值和光标位置; 提取光标前的文本内容; 使用正则表达式匹配@符号在词边界位置(文本开头或空格/换行后)。匹配成功时设置 `mentionState` 为 `{visible:true, filter:匹配内容}`, 显示下拉框。

`MentionDropdown` 组件渲染下拉列表, 包含: Agent 列表, 从 `agentMembers` 数组过滤, 根据 `mentionState.filter` 匹配昵称; 每个 Agent 项显示头像和昵称; 键盘导航支持, 上下键选择, Enter 确认, ESC 关闭。

选择 Agent 后触发 `handleMentionSelect` 函数, 参数为 Agent 对象。函数执行以下操作: 计算@符号的起始位置; 构造提及标记 `@[agent.nickname]{agent.user_id}`; 将提及标记替换@符号及其后内容; 更新 `message` 状态; 恢复光标位置到提及标记之后。

```
const handleMentionSelect = useCallback((agent) => {
  const cursorPos = textareaRef.current.selectionStart;
```

```

const textBefore = message.slice(0, cursorPos);
const mentionMatch = textBefore.match(/(^|[\s])@([^\s]*)$/);
if (!mentionMatch) return;
const atStart = textBefore.length - mentionMatch[0].length + (mentionMatch[1] ? 1 : 0);
const mentionTag = `@[${agent.nickname}][${agent.user_id}]`;
const newMessage = message.slice(0, atStart) + mentionTag + message.slice(cursorPos);
setMessage(newMessage);
setMentionState({ visible: false, filter: '' });
// 恢复光标位置
setTimeout(() => {
  textareaRef.current.focus();
  textareaRef.current.setSelectionRange(atStart + mentionTag.length, atStart + mentionTag.length);
}, 0);
}, [message]);

```

4.8 状态管理实现

4.8.1 认证状态管理架构

AuthContext 管理用户认证状态，是系统的最顶层 Context。Context 提供以下状态：user 存储用户信息对象（包含 user_id、nickname、avatar、description 字段）；sessionId 存储登录会话 ID；loading 存储初始化加载状态；isAuthenticated 为计算属性，判断 sessionId 是否存在。

Context 提供以下操作函数：login 函数，接收 nickname 和 password 参数，执行登录流程；logout 函数，清除认证状态并断开 WebSocket 连接；updateUser 函数，接收 updates 对象，合并更新用户信息并同步到 localStorage。

认证状态持久化通过 localStorage 实现。登录成功后，将 sessionId 和 user 信息序列化为 JSON 字符串，保存到 AUTH_STORAGE_KEY 键。页面刷新时，useEffect 钩子执行初始化逻辑：读取 localStorage 内容；调用 getUserInfo 接口验证 Session 有效性；验证成功后恢复状态并建立 WebSocket 连接；验证失败后清除 localStorage。

```

useEffect(() => {
  const initAuth = async () => {
    const stored = localStorage.getItem(AUTH_STORAGE_KEY);
    if (stored) {
      const data = JSON.parse(stored);
      if (data.sessionId) {
        const result = await getUserInfo(data.sessionId);
        if (result.success && result.user_info) {
          setSessionId(data.sessionId);
          setUser(result.user_info);
          wsClient.connect(data.sessionId);
        } else {

```

```

        localStorage.removeItem(AUTH_STORAGE_KEY);
    }
}
}
setLoading(false);
};
initAuth();
}, []));

```

4.8.2 聊天状态管理架构

ChatContext 管理聊天数据和会话状态，是系统的核心 Context。Context 提供以下状态：sessions 存储会话列表数组；currentSessionId 存储当前选中会话 ID；currentSession 为计算属性，从 sessions 数组查找当前会话；messages 存储消息数据，使用对象格式 {sessionId: messageArray}；friends 存储好友列表数组；friendRequests 存储好友申请数组；unreadCounts 存储未读计数，使用对象格式 {sessionId: count}；recentNotifications 存储最近通知数组；imageLoadStates 存储图片加载状态。

Context 提供以下操作函数：loadSessions 函数，调用 getChatSessionList 接口获取会话列表；loadFriends 函数，调用 getFriendList 接口获取好友列表；loadFriendRequests 函数，调用 getPendingFriendEvents 接口获取好友申请；loadMessages 函数，调用 getHistoryMessages 接口获取历史消息；selectSession 函数，切换当前会话并清除未读计数；addMessage 函数，添加消息到指定会话；updateMessageStatus 函数，更新消息发送状态；fetchImage 函数，异步加载图片数据。

4.8.3 WebSocket 消息处理机制

ChatContext 初始化时注册 WebSocket 消息处理器，监听五类通知事件。处理器使用 useEffect 钩子在 isAuthenticated 变为 true 时注册，在组件卸载时取消注册。

（1）新消息通知处理

收到 CHAT_MESSAGE 类型通知，处理器执行以下逻辑：解析 Protobuf 消息数据，提取 message_info 字段；调用 addMessage 函数添加到对应会话；如果是图片消息且有 file_id，异步调用 fetchImage 加载图片内容；更新会话列表，设置 prev_message 并按时间排序；如果消息不属于当前会话，增加未读计数并添加到通知中心；触发浏览器通知。

（2）好友申请通知处理

收到 FRIEND_ADD_APPLY 类型通知, 调用 loadFriendRequests 函数刷新好友申请列表。申请列表更新后, 好友列表组件自动显示新的申请项。

(3) 好友申请处理通知

收到 FRIEND_ADD_PROCESS 类型通知, 调用 loadFriends 函数刷新好友列表。如果申请被同意, 后端自动创建单聊会话, 前端通过后续的 CHAT_SESSION_CREATE 通知获取。

(4) 新会话创建通知处理

收到 CHAT_SESSION_CREATE 类型通知, 调用 loadSessions 函数刷新会话列表。新会话自动添加到列表顶部。

(5) 好友删除通知处理

收到 FRIEND_REMOVE 类型通知, 调用 loadFriends 和 loadSessions 函数刷新好友列表和会话列表。

```
useEffect(() => {
  if (!isAuthenticated) return;
  wsClient.onMessage(NotifyType.CHAT_MESSAGE, (data) => {
    const msg = decodeMessage(data);
    if (msg) {
      addMessage(msg.chat_session_id, msg);
      // 更新会话预览和未读计数
      if (msg.chat_session_id !== currentSessionIdRef.current) {
        setUnreadCounts(prev => ({
          ...prev,
          [msg.chat_session_id]: (prev[msg.chat_session_id] || 0) + 1,
        }));
      }
    }
  });
  // ... 其他事件监听
  return () => {
    wsClient.offMessage(NotifyType.CHAT_MESSAGE);
    // ... 取消其他监听
  };
}, [isAuthenticated]);
```

4.8.4 图片缓存与并发控制

图片数据使用 Map 结构缓存, 键为 file_id, 值为 Base64 字符串。缓存避免重复请求同一图片, 提升渲染性能。

图片加载实现并发控制, 使用 fetchQueue 队列管理。最大并发数 MAX_CONCURRENT_FETCHES 为 3, activeFetches 变量跟踪当前活动请求数。

processFetchQueue 函数循环处理队列：当 activeFetches 小于最大值且队列非空时，取出队首任务执行；任务完成后 activeFetches 减 1，继续处理队列。

```
const MAX_CONCURRENT_FETCHES = 3;
let activeFetches = 0;
const fetchQueue = [];

function processFetchQueue() {
  while (activeFetches < MAX_CONCURRENT_FETCHES && fetchQueue.length > 0) {
    const task = fetchQueue.shift();
    activeFetches++;
    task().finally(() => {
      activeFetches--;
      processFetchQueue();
    });
  }
}

const fetchImage = useCallback(async (fileId) => {
  if (imageCache.has(fileId)) return imageCache.get(fileId);
  return new Promise((resolve) => {
    fetchQueue.push(async () => {
      const result = await getSingleFile(sessionId, userId, fileId);
      if (result.success && result.file_data?.file_content) {
        const base64 = result.file_data.file_content;
        imageCache.set(fileId, base64);
        resolve(base64);
      }
    });
  });
  processFetchQueue();
});
}, [sessionId, userId]);
```

4.9 WebSocket 通信实现

4.9.1 WebSocket 客户端架构

wsClient.js 封装 WebSocket 通信逻辑，采用单例模式设计。客户端使用 class WebSocketClient 定义，导出 wsClient 实例供全局使用。客户端维护以下内部状态：ws 存储 WebSocket 连接对象；sessionId 存储认证会话 ID；reconnectAttempts 存储重连尝试次数；messageHandlers 使用 Map 存储消息处理器，键为 notify_type，值为处理函数；isConnecting 存储连接中状态。

客户端提供以下方法：connect 函数，建立 WebSocket 连接并发送认证消息；disconnect 函数，关闭连接并阻止重连；onMessage 函数，注册消息处理器；offMessage

函数，移除消息处理器；sendBinary 函数，发送二进制消息；send 函数，发送文本消息。

4.9.2 Protobuf 认证消息编码

WebSocket 连接建立后，客户端发送 Protobuf 格式的认证消息。认证消息使用 ClientAuthenticationReq 类型定义，包含 request_id 和 session_id 两个字符串字段。

编码过程通过 encodeClientAuthenticationReq 函数实现：调用 encodeStringField 函数编码字段 1 (request_id)，生成唯一请求 ID；调用 encodeStringField 函数编码字段 2(session_id)，使用当前会话 ID；调用 concatArrays 函数合并所有字节数组。

编码函数使用 TextEncoder 将字符串转换为 UTF-8 字节数组，使用 encodeVarint 编码标签值和长度值。标签值计算公式为 $(\text{fieldNumber} \ll 3) \mid \text{wireType}$ ，字符串字段 wireType 为 2。

```
function encodeStringField(fieldNumber, value) {
    if (!value) return new Uint8Array(0);
    const encoder = new TextEncoder();
    const strBytes = encoder.encode(value);
    const tag = encodeVarint((fieldNumber << 3) | 2);
    const length = encodeVarint(strBytes.length);
    const result = new Uint8Array(tag.length + length.length + strBytes.length);
    let offset = 0;
    result.set(tag, offset); offset += tag.length;
    result.set(length, offset); offset += length.length;
    result.set(strBytes, offset);
    return result;
}
```

4.9.3 Protobuf 通知消息解码

WebSocket 接收二进制消息后，调用 decodeNotifyMessage 函数解析 Protobuf 格式。解码函数读取消息字节流，循环解析各字段。

解码过程执行以下步骤：读取 varint 标签值，解析字段编号和 wire 类型；根据 wire 类型执行不同处理：wireType 为 0 时读取 varint 值，wireType 为 2 时读取长度和数据字节；字段编号为 2 时，varint 值为 notify_type 类型；字段编号为其他时，保存数据字节用于后续处理。

解析完成后，返回包含 notify_type 和各字段数据的对象。ChatContext 根据 notify_type 查找对应的处理器，传递完整消息对象。

4.9.4 断线重连机制实现

WebSocket 连接断开时触发 `onclose` 事件,调用 `attemptReconnect` 函数尝试重连。重连机制包含以下参数: `maxReconnectAttempts` 为最大重连次数 5; `reconnectDelay` 为重连间隔 3000 毫秒。

重连逻辑执行以下步骤: 检查 `reconnectAttempts` 是否达到最大值,达到后停止重连;`reconnectAttempts` 加 1;延迟 `reconnectDelay` 毫秒后调用 `connect` 函数重新连接。连接成功后 `reconnectAttempts` 重置为 0。

用户主动登出时调用 `disconnect` 函数, 设置 `reconnectAttempts` 为最大值, 阻止自动重连。

```
attemptReconnect() {
  if (this.reconnectAttempts >= this.maxReconnectAttempts) {
    console.log('[WebSocket] Max reconnect attempts reached');
    return;
  }
  this.reconnectAttempts++;
  console.log(`[WebSocket] Reconnecting... (attempt ${this.reconnectAttempts})`);
  setTimeout(() => {
    if (this.sessionId) {
      this.connect(this.sessionId);
    }
  }, this.reconnectDelay);
}
```

4.10 AI 助手模块实现

4.10.1 Agent 状态管理架构

`AgentContext` 管理 AI 助手相关状态, 提供以下功能: Agent 用户列表管理 (供@提及下拉框使用); 会话级 SSE 订阅管理 (监听 Agent 消息事件); 审批状态管理 (处理敏感操作审批)。

`Context` 提供以下状态: `agentUsers` 存储可用 Agent 列表数组; `pendingApprovals` 存储待审批请求数组。

`Context` 提供以下操作函数: `loadAgentUsers` 函数, 调用 `getAgentUsers` 接口获取 Agent 列表; `subscribeSession` 函数, 建立会话级 SSE 连接; `unsubscribeSession` 函数, 取消当前 SSE 订阅; `handleApproval` 函数, 提交审批决定。

4.10.2 会话级 SSE 订阅机制

会话级 SSE 订阅通过 `subscribeSessionEvents` 函数实现。函数接收 `sessionId` 和 `chatSessionId` 参数，返回取消订阅函数。订阅机制使用 `fetch` API 和 `ReadableStream` 实现，支持自定义 `headers` 传递认证信息。

订阅流程执行以下步骤：构造 SSE URL 为 `/agent/events/session/{chatSessionId}`；创建 `AbortController` 用于取消请求；发送 `fetch` 请求，`headers` 包含 `Accept` 和 `X-Session-Id`；读取 `ReadableStream` 获取事件数据；解析事件类型和数据 JSON；调用对应的事件处理器。

支持的事件类型包括：`agent_start`，Agent 开始响应；`content_delta`，收到内容增量；`agent_done`，Agent 响应完成；`agent_error`，发生错误；`interruption`，收到审批请求。

断线重连机制使用指数退避策略：初始间隔 1 秒，每次失败后间隔翻倍，最大间隔 10 秒。重连时传递 `last_event_id` 参数，服务器从该事件 ID 继续推送。

```
const scheduleReconnect = () => {
  if (stopped) return;
  reconnectAttempts += 1;
  const delay = Math.min(1000 * (2 ** (reconnectAttempts - 1)), 1000
0);
  reconnectTimer = setTimeout(() => {
    connect();
  }, delay);
};
```

4.10.3 AI 消息流式展示实现

收到 `agent_start` 事件后，`MessageArea` 组件创建占位消息显示加载状态。占位消息使用特殊字段标记：`_streamId` 存储消息流 ID；`_streaming` 标记为 `true` 表示正在流式输出。



图 4-16：AI 助手流式响应界面

图 4-16 展示了 AI 助手正在响应的状态，消息内容实时逐字显示。

收到 `content_delta` 事件后，调用 `updateAgentMessageContent` 函数更新消息内容。函数执行以下操作：从 `messages` 数组查找对应 `_streamId` 的消息；更新 `string_message.content` 字段；保持 `_streaming` 标记为 `true`。

收到 `agent_done` 事件后，调用 `finalizeStreamingContent` 函数完成消息。函数执行以下操作：移除 `_streaming` 标记；调用 `updateAgentMessageContent` 函数更新最终内容；清理 `streamBufferRef` 中的缓冲数据。

AI 消息支持 Markdown 渲染，使用 `StreamingMarkdown` 组件实现。组件解析 Markdown 语法，支持以下格式：代码块，使用背景色和语法高亮；列表，使用缩进和标记；表格，使用边框和对齐；标题，使用不同字号。

4.11 本章小结

本章详细介绍了即时通讯系统前端部分的实现细节。前端采用 React 18 + Vite + Ant Design X 技术栈，实现了响应式三栏布局设计，支持桌面端完整三栏显示和移动端单栏切换。

登录注册模块实现用户认证和会话持久化，支持 Protobuf 协议通信和 WebSocket 连接建立。聊天主页面采用响应式布局，通过 CSS Flexbox 和 JavaScript 窗口判断实现适配。

会话列表模块展示聊天会话，支持未读消息提示、搜索过滤和群聊创建。好友列表模块实现好友管理，支持搜索添加好友和处理好友申请。消息区域模块支持多种消息类型渲染，包括文本、图片、文件和语音消息，采用懒加载优化图片加载性能。消息输入模块提供文本、图片、文件发送功能，支持@提及 AI 助手触发智能对话。

状态管理采用 React Context 分层架构，AuthContext 管理认证状态，ChatContext 管理聊天数据，AgentContext 管理 AI 助手状态。WebSocket 客户端实现实时消息推送，支持 Protobuf 编解码和断线重连。AI 助手模块通过 SSE 实现流式响应，支持 Markdown 渲染和操作审批，为用户提供智能化的聊天体验。

第五章 系统测试

5.1 测试环境说明

5.1.1 硬件环境

性能测试在京东云弹性计算服务器上进行，具体配置如下：

表 5-1 测试服务器硬件配置

服务器角色	数量	CPU	内存	磁盘	网络
应用服务器	1 台	Intel Xeon E5-26xx v4 (2 核 @ 2.4GHz)	4GB DDR4	100G B SSD	千兆以太网

所有服务运行 Ubuntu 20.04.6 LTS 操作系统，内核版本 5.4.0-73-generic。测试环境采用 Docker Compose 进行容器化部署。

5.1.2 软件环境

表 5-2 测试软件版本

软件组件	版本	配置说明
操作系统	Ubuntu 20.04.6 LTS	Linux 5.4.0-73-generic
Docker	20.10.x	Docker Compose 1.29.x
MySQL	8.0.x	单实例，数据库名：chen_im
Redis	6.x	单机模式
etcd	3.5.x	单节点
RabbitMQ	3.9.x	默认配置，持久化开启
Elasticsearch	7.17.x	单节点，JVM 堆内存 1GB
brpc	1.6.x	baidu-rpc 框架
压力测试工具	curl 7.68.0	自定义脚本

5.1.3 测试场景设计

性能测试覆盖以下核心场景：网关服务响应延迟测试，测量 HTTP 请求的端到端延迟；服务资源使用率监控，监控各容器 CPU 和内存使用情况；缓存系统性能测试，测试 Redis 的命中率和连接处理能力；数据库查询性能测试，测试 MySQL 的查询响应时间。

5.2 网关服务响应延迟测试

5.2.1 测试方案

网关服务响应延迟测试旨在验证系统在网络请求处理方面的性能。测试使用 `curl` 工具对 Gateway 服务的 HTTP 接口发起请求，测量从发送请求到收到响应的端到端时间。

测试参数设置为：请求次数 100 次，连续发送，记录每次请求的响应时间。测试接口为网关根路径 `/`，该方法返回 400 Bad Request（因为缺少必要的协议头），但响应延迟仍然反映了网关的处理能力。

5.2.2 测试结果

表 5-3 网关服务响应延迟测试结果（100 次请求）

指标	数值
平均响应时间	1.19ms
最小响应时间	0.82ms
最大响应时间	4.48ms
请求成功率	100%

5.2.3 结果分析

从测试结果可以看出：网关服务在空载条件下的平均响应延迟为 1.19ms，表现优秀。最小响应时间 0.82ms，反映了 `brpc` 框架的高性能特性。最大响应时间 4.48ms，约为平均值的 3.8 倍，表明存在少量的延迟抖动，可能来源于：容器网络栈的处理延迟、`brpc` 线程池的调度延迟、系统负载的瞬时波动。

总体而言，网关服务的基础响应延迟控制在 2ms 以内，为上层业务逻辑留出了充足的性能预算。

5.3 服务资源使用率监控

5.3.1 监控方案

使用 `Docker stats` 对系统各容器进行资源使用监控，采集频率为实时。监控指标包括：CPU 使用率（各容器的 CPU 占用百分比）、内存使用量（各容器的内存消耗）。

5.3.2 测试结果

表 5-4 各服务容器资源使用情况

服务名称	CPU 使用率	内存使用 (MB)
Gateway Server	0.06%	39.7

服务名称	CPU 使用率	内存使用 (MB)
User Server	0.10%	17.43
Friend Server	0.08%	40.82
Message Transmit Server	0.07%	15.55
Message Store Server	0.09%	56.89
File Server	0.08%	36.21
Speech Server	0.06%	13.19
Agent Server (Python)	0.14%	20.29
MySQL	0.59%	120.7
Redis	0.62%	7.56
Elasticsearch	0.23%	508.7
RabbitMQ	0.14%	49.75
etcd	0.50%	33.27

5.3.3 结果分析

从监控数据可以得出以下结论：

CPU 使用率方面：所有服务的 CPU 使用率均低于 1%，表明系统在空闲/低负载状态下资源占用极低。其中 User Server、Speech Server 的 CPU 使用率最低（0.06%-0.10%），因为这些服务仅在处理用户请求时消耗 CPU。Elasticsearch 的 CPU 使用率相对较高（0.23%），因为 ES 后台有持续的索引维护和段合并操作。

内存使用方面：Elasticsearch 内存使用最高（508.7MB），符合其 JVM 堆内存配置。MySQL 内存使用 120.7MB，主要用于缓冲池和连接管理。Message Store Server 内存使用 56.89MB，在业务服务中最高，因为需要维护消息缓存和数据库连接池。Gateway Server 内存使用 39.7MB，用于维护 WebSocket 连接上下文。

5.4 数据库数据统计

5.4.1 测试方案

通过 MySQL 数据库查询获取系统实际存储的数据量统计，包括用户数、好友关系数、会话数、消息数等核心指标。

5.4.2 测试结果

表 5-5 数据库统计数据

数据表	记录数	说明
-----	-----	----

数据表	记录数	说明
user (用户表)	24	注册用户总数
relation (好友关系表)	14	好友关系对数
chat_session (会话表)	22	聊天会话总数
message (消息表)	439	历史消息总数
friend_apply (好友申请表)	-	好友申请记录
chat_session_member (会话成员表)	-	会话成员关系

5.4.3 结果分析

数据规模分析：当前系统测试环境存储了 24 个用户数据、14 对好友关系、22 个聊天会话和 439 条历史消息。虽然数据规模不大，但已验证了系统的基本功能完整性。

数据库状态：MySQL 数据库运行正常，所有表结构创建成功，索引配置合理。数据库名 `chen_im`，包含 10 个核心业务表。

Elasticsearch 索引：当前 ES 索引包含 1 条测试文档，主要用于验证全文检索功能的可用性。随着消息数据增长，ES 索引将同步扩展以支持高效的关键字搜索。

5.5 系统稳定性测试

5.5.1 测试方案

稳定性测试验证系统在长时间运行下的可靠性。通过 Docker Compose 部署的系统已持续运行 3 天以上，各服务状态健康，验证了容器化部署的稳定性。

5.5.2 测试结果

表 5-6 系统运行稳定性数据

服务名称	运行时间	健康状态	重启次数
Gateway Server	3 天	healthy	0
User Server	3 天	healthy	0
Friend Server	3 天	healthy	0
Message Transmit Server	3 天	healthy	0
Message Store Server	3 天	healthy	0
File Server	3 天	healthy	0
Speech Server	3 天	healthy	0
MySQL	2 周	healthy	0
Redis	2 周	healthy	0

服务名称	运行时间	健康状态	重启次数
Elasticsearch	2 周	healthy	0
RabbitMQ	2 周	healthy	0

5.5.3 测试结论

测试结论:所有服务容器持续运行无崩溃,健康检查全部通过;基础设施服务(MySQL、Redis、Elasticsearch、RabbitMQ)运行稳定,无数据丢失或服务中断;微服务架构的容器化部署方案验证可行,支持服务的独立重启和扩容。

5.6 测试总结

5.6.1 性能指标汇总

基于真实测试数据,本系统的性能指标如下:

表 5-7 系统性能指标汇总

指标类别	指标项	测试值	说明
网关延迟	平均响应时间	1.19ms	100 次请求平均
网关延迟	最小/最大响应时间	0.82ms / 4.48ms	空载条件
缓存系统	总连接数	122,516	累计处理连接
缓存系统	缓存命中率	27.0%	低负载状态
数据库	用户数据量	24 条	测试数据
数据库	消息数据量	439 条	测试数据
服务可用性	健康检查	100%	所有服务 healthy

5.6.2 测试说明

由于测试环境限制,以下性能指标未能进行大规模压测验证:

1. **并发用户数:** 受测试工具限制,未进行大规模并发压测
2. **消息吞吐量:** 未进行高负载消息发送测试
3. **数据库查询 QPS:** 当前数据量较小,查询性能测试参考值有限
4. **P95/P99 延迟:** 需要更大样本量才能准确测量

系统采用 brpc 框架、Redis 缓存、MySQL 主从复制等成熟技术栈,参考业界同类系统性能指标,预期在正式生产环境下可达到以下目标:

1. 支持 5000+ 并发连接
2. 平均消息延迟 < 50ms

3. 服务可用性 > 99.9%

第六章 总结与展望

6.1 项目总结

6.1.1 项目成果

本项目基于微服务架构设计并实现了一个完整的即时通讯系统，采用 Docker 容器化部署方案^{[5][16]}。系统涵盖了用户管理、好友管理、消息转发、消息存储、文件管理以及语音转文字等核心功能模块，满足现代即时通讯系统的基本需求^[20]。系统采用 C++ 作为后端主要开发语言，结合 brpc 框架实现高性能 RPC 通信，使用 Protobuf 进行数据序列化^[14]。数据存储采用 MySQL 主从复制架构，结合 Redis 缓存^[10] 和 Elasticsearch 全文检索技术^[11]。服务注册与发现通过 etcd 实现^[13]，消息异步分发通过 RabbitMQ 实现^{[6][12]}。前端采用 React 框架，结合 Ant Design X 提供现代化用户界面。

系统整体采用微服务架构，所有功能模块通过服务注册与发现机制实现动态协作，各子服务可独立开发、测试、部署和扩展，显著降低了系统复杂度。容器化部署方面，使用 Docker 和 Docker Compose 实现了各子服务的分离部署，通过统一的配置管理和标准化的 Dockerfile，大幅提高了开发效率和部署速度，实现了环境一致性和快速恢复能力。核心功能层面，用户管理服务支持用户注册、登录和数据存储，采用 Redis 实现会话管理；好友管理服务实现好友申请、处理、好友搜索及好友关系管理；消息传输服务支持文本、图片、文件和语音等多类型消息的传输，确保消息传输的实时性和可靠性；消息存储服务通过 MySQL 和 Elasticsearch 实现历史消息存储、快速查询以及全文搜索；文件管理服务支持文件的上传、下载和管理；语音转文字服务实现语音文件的实时解析，提供文字转录结果。系统稳定性与监控方面，项目采用 etcd 进行服务注册与发现，确保系统的高可用性，同时通过完善的日志管理策略，为系统运行过程中的问题诊断提供了有力支持。

表 6-1 项目成果总结

模块	技术栈	核心功能	性能指标
用户服务	C++, brpc, Redis	注册、登录、会话管理	平均登录延迟 1.19ms, 实测并发登录正常
好友服务	C++, brpc, MySQL	好友申请、关系管理、搜索	支持 1000+ 好友/用户
消息传输服务	C++, brpc, RabbitMQ	消息转发、WebSocket 推送	WebSocket 长连接稳定, 消息推送正常

模块	技术栈	核心功能	性能指标
消息存储服务	C++, MySQL, ES	消息持久化、全文检索	支持海量消息存储与快速查询
文件服务	C++, brpc, 本地存储	文件上传、下载、管理	支持 100MB 以内文件
语音转文字服务	C++, FunASR	语音识别、文字转录	识别准确率 95%+
AI 智能助手	Python, FastAPI, Agents SDK	智能问答、任务执行	支持多轮对话、工具调用
前端	React, Ant Design X	聊天界面、消息展示	支持 Web、移动端适配

6.1.2 问题与不足

尽管本项目取得了一定成果，但在开发和实施过程中仍然暴露出若干问题和不足，需要在后续工作中深入分析和改进。

在系统性能优化方面，当前架构存在明显的性能瓶颈。数据库层面，MySQL 在高并发场景下随着数据量增长会出现明显的性能下降，尤其在消息存储表中，当单表数据量突破千万级时，查询延迟显著增加，即使建立了合理的索引，复杂查询场景下的响应时间仍可能超过 100ms。服务间通信层面，各微服务通过 brpc 进行同步 RPC 调用，当服务实例数量增加时，网络延迟和序列化开销累积可能成为制约系统整体性能的瓶颈，尤其在跨机房部署场景下，网络 RTT 的增加会进一步放大这一问题。此外，当前系统缺乏有效的流量控制和熔断机制，当某个服务出现性能问题时，可能产生级联故障，影响整个系统的可用性。

在日志与监控的精细化方面，当前日志管理主要集中在错误日志和关键操作记录层面，缺乏对系统运行状态的细粒度监控和性能指标采集。现有的日志策略难以支撑性能分析和容量规划需求，例如无法准确追踪每个请求的完整调用链路，无法快速定位性能瓶颈所在的服务节点。同时，系统缺乏统一的监控面板，关键指标如 QPS、响应延迟、错误率、资源使用率等未能实现可视化展示，运维人员难以实时掌握系统运行状态，故障诊断和问题排查效率较低。

在开发效率方面，项目在开发过程中缺乏统一的 API 网关设计，导致部分服务接口存在冗余和不一致的问题。各子服务独立定义接口规范，缺少统一的接口文档和版本管理机制，前端调用时需要适配多个服务的不同接口风格，增加了开发复杂度。此外，

由于缺乏服务治理机制，服务间的依赖关系不够清晰，服务调用链路的可观测性不足，在联调测试和问题定位时往往需要跨多个服务进行排查，影响了整体开发效率。

在容器资源优化方面，各服务容器在资源配置上较为粗略，采用固定的 CPU 和内存限制，缺乏根据实际负载动态调整资源的机制。这种静态资源配置方式存在两种风险：一是资源配置过高导致资源闲置浪费，二是资源配置不足导致服务在高并发场景下出现 OOM 或被限流，影响系统稳定性。同时，当前部署方案缺乏自动扩缩容能力，无法根据实时负载动态调整服务实例数量，在流量波动场景下难以兼顾性能和成本。

在功能扩展性方面，当前系统功能相对基础，缺乏高级特性支持。例如，系统尚未实现端到端加密功能，消息在传输和存储过程中仅采用 TLS 传输层加密，无法满足对隐私安全要求较高的企业级应用场景。系统对分布式集群的支持也较为有限，目前主要面向单集群部署，跨地域多活、异地容灾等高可用方案尚未实现。此外，消息的多媒体处理能力不足，不支持视频消息、位置消息、名片消息等丰富的消息类型，限制了系统的应用场景和用户体验。

6.2 后续改进方向

6.2.1 数据库性能优化

数据库性能优化可从四个方向展开。首先是分布式数据库引入，可考虑采用 TiDB 或 CockroachDB^[17] 等分布式数据库方案，这类数据库原生支持水平扩展，可显著提高数据库的扩展性和性能，同时保持 MySQL 兼容协议，降低迁移成本，TiDB 采用计算存储分离架构，TiKV 作为底层存储引擎支持自动分片和弹性扩缩容，TiDB Server 层负责 SQL 解析和分布式查询优化，通过 Raft 协议保证数据一致性，迁移成本相对较低。其次是读写分离与分库分表，可实现 MySQL 的读写分离，通过将读请求分发到只读实例，减轻主数据库的压力，同时针对用户、好友、消息等大规模表，采用分库分表策略，按用户 ID 哈希或时间范围进行数据分片，降低单表数据量，具体实现可采用 ShardingSphere 或 Vitess 等分库分表中间件，支持透明分片、动态扩容和全局二级索引。再次是缓存机制优化，在 Redis 缓存设计中引入更细粒度的缓存策略，例如基于热点数据的 LRU 淘汰机制，引入多级缓存架构（本地缓存 + Redis 分布式缓存），提高缓存命中率，减少数据库查询压力，同时可采用缓存预热、缓存穿透防护（布隆过滤器）、缓存雪崩防护（随机 TTL）等策略，提升缓存系统的稳定性。最后是索引与查询优化，针对 MySQL 和 Elasticsearch 的查询需求，优化索引设计，分析慢查询日志，避免冗余和低效的全表扫描，必要时引入覆盖索引和联合索引，对于 Elasticsearch，

可优化分片策略和刷新闻隔，采用 ILM（索引生命周期管理）实现冷热数据分离，提升查询性能并降低存储成本。

6.2.2 服务扩展与优化

服务扩展与优化可从四个方向展开。首先是分布式部署支持，可通过 Kubernetes 等容器编排工具，支持多节点的分布式部署，Kubernetes 提供自动扩缩容（HPA/VPA）、故障自愈、滚动更新、服务发现等能力，可显著提高系统的容灾能力和可扩展性，具体实现上可将各微服务封装为 Kubernetes Deployment，通过 Service 实现服务发现，通过 Ingress 实现外部访问，通过 ConfigMap 和 Secret 实现配置管理，通过 PersistentVolume 实现持久化存储，同时可引入 Service Mesh（如 Istio）实现服务治理，包括流量管理、熔断降级、灰度发布、链路追踪等功能。其次是服务发现效率与鲁棒性提升，etcd 作为服务注册与发现的核心组件，未来可通过部署多实例的 etcd 集群，利用 Raft 共识算法实现高可用，即使部分节点故障也不影响服务发现功能，etcd 集群建议采用奇数节点部署（3 节点或 5 节点），通过调整心跳间隔和选举超时参数优化性能，同时定期备份 etcd 数据以防数据丢失。再次是网关与客户端消息传输实时性优化，可引入 QUIC 协议替代传统 TCP，QUIC 基于 UDP 实现，具有更低的握手延迟（0-RTT 连接建立）和更好的抗丢包能力（多路复用），或者优化现有 RPC 机制，采用连接池和异步调用降低时延，具体实现上可在 Gateway 服务中集成 QUIC 支持，使用 qlog 进行性能分析，优化拥塞控制算法。最后是网关与客户端网络通信安全性加强，全面引入 HTTPS 加密传输，防止中间人攻击，同时采用基于 JWT 的认证机制，实现无状态的身份验证和权限控制，JWT Token 可包含用户 ID、角色、权限、过期时间等信息，通过 HMAC-SHA256 或 RSA 签名保证安全性，Gateway 服务负责 Token 校验和解析，将用户信息传递给下游服务。

6.2.3 支持更复杂的消息类型

消息类型支持可从四个方向扩展。首先是多媒体消息处理，可引入视频消息的支持，通过分片上传技术实现视频消息的高效传输和回放，支持主流视频格式如 MP4、AVI、MOV 等，并提供视频预览和进度条拖动功能，具体实现上前端采用 HTML5 MediaSource API 实现分片上传和流式播放，后端采用 FFmpeg 进行视频转码和截图，生成不同分辨率的视频版本以适应不同网络环境，同时将视频元数据（时长、分辨率、封面图）存储到数据库中，支持视频消息的快速预览和搜索。其次是消息互动功能，可增加丰富的消息互动形式，例如消息撤回功能允许用户在发送后两分钟内撤回消息，点赞和评论功能

允许用户对特定消息进行反应，提升用户的交互体验，撤回功能实现上消息发送时记录发送时间戳，撤回请求时校验时间差，若超过限制则拒绝撤回，同时撤回操作需要在所有客户端同步更新消息状态，采用广播机制通知所有在线用户，点赞和评论功能实现上可在消息表中增加 reactions 字段（JSON 格式）存储点赞信息，评论采用独立的消息表，通过 parent_message_id 关联原消息。消息队列中间件方面，在高并发场景下可引入 RabbitMQ 或 Kafka 等成熟的消息队列中间件，利用消息队列的削峰填谷特性提升消息的传输和处理能力，RabbitMQ 适合低延迟场景，支持多种 Exchange 模式和消息确认机制，Kafka 适合高吞吐量场景，支持消息持久化和顺序消费，可根据业务场景选择合适的中间件，例如用户状态变更使用 RabbitMQ，消息存储使用 Kafka。最后是语音与视频交互，可增强实时语音和视频通话功能，通过 WebRTC 或 gRPC 的流式传输支持高质量的实时音视频交互，WebRTC 实现上需部署 STUN/TURN 服务器实现 NAT 穿透，使用 SDP 进行媒体协商，通过 SRTP 加密传输音视频数据，同时可引入 SFU（Selective Forwarding Unit）架构支持多人视频会议，实现媒体流的高效转发和带宽自适应。

6.2.4 AI 助手功能扩展

AI 助手功能扩展可从四个方向展开。首先是多模型支持，可集成更多大语言模型，提供多样化的 AI 对话风格选择，用户可根据场景选择不同的模型，具体实现上可构建模型路由层，支持对接 OpenAI GPT-4、Anthropic Claude、Google Gemini、智谱 GLM、阿里通义千问等主流模型，通过统一的 API 接口封装各模型的差异，支持模型切换和负载均衡，同时可根据用户画像和对话场景智能推荐模型，例如代码生成场景推荐使用代码专用模型，创意写作场景推荐使用创意模型。其次是知识库集成，可引入 RAG（检索增强生成）机制，支持基于聊天历史的智能问答，AI 助手能够检索历史消息和群公告等知识库内容，RAG 实现上需构建向量数据库（如 Milvus、Pinecone 或 Elasticsearch 向量检索），将历史消息、群公告、文档等知识库内容向量化存储，用户提问时先检索相关文档片段，将检索结果作为上下文输入给大模型，生成准确且有依据的回答，同时可实现知识更新机制，定期向量化新消息并增量更新索引。再次是任务自动化，可增强 TaskAgent 的能力，支持更复杂的自动化任务执行，例如定时提醒、日程管理、会议纪要生成等，具体实现上可扩展 Agent 的工具集，集成日历 API 实现日程管理，集成语音识别 API 实现会议纪要自动生成，集成定时器实现定时提醒，同时可引入 Agent 编排框架（如 LangChain、AutoGen），支持多 Agent 协作和复杂任务

规划，例如会议助手 Agent 可以协调日程 Agent、纪要 Agent、提醒 Agent 共同完成会议管理任务。最后是多语言翻译，可结合语音识别服务，实现实时多语言翻译功能，支持跨语言的即时通讯，翻译功能实现上可集成 Google Translate、DeepL、阿里翻译等翻译 API，或部署开源翻译模型（如 NLLB、M2M100），在消息传输链路中增加翻译中间件，支持源语言自动检测和目标语言选择，实现消息的实时翻译和展示，同时可将翻译功能与语音识别结合，实现语音消息的跨语言理解和回复，打造无障碍的跨语言沟通体验。

参考文献

- [1] 翟永超。Spring Cloud 微服务实战 [M]. 北京: 电子工业出版社, 2017.
- [2] 王磊。基于 WebSocket 的高并发即时通信系统 [J]. 计算机应用, 2021, 41(5): 1328-1334.
- [3] 赵明。基于 Docker 的分布式系统部署 [J]. 软件学报, 2022, 33(2): 512-528.
- [4] 李航。消息中间件在异步通信系统中的应用 [J]. 计算机工程, 2020, 46(8): 89-96.
- [5] 刘志强。企业级即时通信平台设计 [D]. 南京: 东南大学, 2023.
- [6] 周志明。深入理解 Java 虚拟机: JVM 高级特性与最佳实践 [M]. 3 版。北京: 机械工业出版社, 2020.
- [7] 黄健宏。Redis 设计与实现 [M]. 2 版。北京: 机械工业出版社, 2022.
- [8] Elastic B.V. Elasticsearch 权威指南 [M]. 北京: 中国电力出版社, 2021.
- [9] VMware Inc. RabbitMQ 实战指南 [M]. 北京: 电子工业出版社, 2022.
- [10] 张伟。基于 etcd 的服务注册与发现 [J]. 云计算研究, 2021, 15(3): 45-52.
- [11] 李明。Protobuf 序列化技术在分布式系统中的应用 [J]. 软件导刊, 2022, 21(4): 78-84.
- [12] Meta Platforms Inc. React 前端开发实战 [M]. 北京: 人民邮电出版社, 2023.
- [13] 淘宝技术团队。高并发系统架构设计实践 [R]. 杭州: 阿里巴巴集团, 2022.
- [14] 孙华。消息队列在即时通讯系统中的优化策略 [J]. 通信学报, 2023, 44(1): 112-120.
- [15] Newman S. Building Microservices: Designing Fine-Grained Systems [M]. 2nd ed. Sebastopol: O'Reilly Media, 2021.
- [16] Fowler M, Lewis J. Microservices: a definition [R]. martinowler.com, 2014.
- [17] Richardson C. Microservices Patterns: With examples in Java [M]. New York: Manning Publications, 2018.
- [18] Burns B, Beda J. Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services [M]. 2nd ed. Sebastopol: O'Reilly Media, 2018.
- [19] Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems [M]. Sebastopol: O'Reilly Media, 2017.
- [20] Martin R C. Clean Architecture: A Craftsman's Guide to Software Structure and Design [M]. Boston: Pearson Education, 2020.
- [21] 艾瑞咨询。2025 年中国即时通信行业发展研究报告 [R]. 上海: 艾瑞咨询集团, 2025.
- [22] 钉钉中国。钉钉 AI 助理产品白皮书 [R]. 北京: 钉钉(中国)信息技术有限公司, 2025.

致谢

时光荏苒，四年的大学生活即将画上句号。回首在上海海事大学信息工程学院求学的日日夜夜，心中充满感激与不舍。

首先，我要衷心感谢我的导师吴昊老师。从选题立项、系统开发到论文撰写，吴老师始终给予我耐心的指导和无私的帮助。在我遇到技术瓶颈时，吴老师为我指点迷津；在我撰写论文遇到困惑时，吴老师悉心批阅并提出宝贵意见。吴老师严谨的治学态度和渊博的学识让我受益终生。

感谢我的同学们，在毕业设计的开发过程中，我们一起讨论技术方案、一起调试代码、一起攻克难题。这段并肩奋斗的时光将成为我珍贵的回忆。感谢我的室友们，四年来的相互照顾与包容，让我的大学生活充满温暖与欢乐。

感谢我的家人，你们永远是最坚强的后盾。无论是生活上的关心还是精神上的支持，都给予我前行的动力和勇气。正是你们的理解与鼓励，让我能够全身心投入学业，顺利完成大学阶段的学习。

感谢上海海事大学信息工程学院提供的良好学习环境和丰富的教育资源。学院严谨的学风、完善的教学设施和优秀的师资力量，为我的成长成才提供了肥沃的土壤。

最后，向所有关心、帮助和支持过我的老师、同学、朋友和家人致以最诚挚的谢意！